

时空可组合性编程范式

中译本（非官方翻译）

Yifan Shi（北京大学） Wei Zhang（北京大学） Tianyi Cui（DeepSeek-AI）

原论文 2026 年 · 中文翻译完成于 2026-08-16

Contents

1	引言	3
1.1	可组合性的两个维度	3
1.2	动机示例	3
1.2.1	插件系统	3
1.2.2	自进化智能体脚手架	4
1.2.3	粗粒度的权宜之计	4
1.3	贡献	4
2	预备知识	5
2.1	效应	5
2.2	共效应	5
2.3	与动态可组合性的关系	6
3	可逆效应与响应式共效应	6
3.1	可逆效应	6
3.1.1	效应上下文	6
3.1.2	可逆效应函数	8
3.1.3	效应的独立性	9
3.2	响应式共效应	11
3.2.1	共效应上下文	11
3.2.2	规范与通知	12
3.2.3	隔离与拦截	12
3.3	上下文范式	13
3.3.1	统一上下文	13
3.3.2	观察等价	13
3.3.3	范式定位	15
4	动态组合演算	16
4.1	组件与纤维	16
4.2	基础演算	17
4.3	进行中的迁移	18
4.3.1	撤回	18
4.3.2	迭代	18
4.3.3	异步	19
4.3.4	失败	19
4.4	元理论	19
4.4.1	保持性	20
4.4.2	时间可组合性	20

4.4.3	空间可组合性	21
4.4.4	进展性	21
4.4.5	汇合性	21
5	实现与案例研究	22
5.1	核心库	22
5.1.1	效应追踪	22
5.1.2	共效应运算	23
5.1.3	组件生命周期	23
5.1.4	上下文访问	24
5.2	组件加载器	24
5.2.1	声明式配置	24
5.2.2	热模块替换	25
5.3	案例研究: Koishi	25
6	讨论	26
6.1	系统边界	26
6.2	服务多路复用	26
6.3	访问控制与沙箱	26
6.4	语言无关性与选择	27
6.5	相互依赖与组件粒度	27
6.6	依赖类型化与版本化	27
6.7	与语言和操作系统的协同设计	27
7	相关工作	28
7.1	效应与共效应系统	28
7.2	编程范式	28
7.3	时间可组合性	29
7.4	空间可组合性	29
8	结论	30

Abstract

现代软件——从插件系统到自进化智能体脚手架——日益需要**动态组合**，但其形式基础仍不成熟。本文识别出动态组合的两个正交维度：**时间可组合性**（组件移除时其副作用被完全、安全地逆转的能力）与**空间可组合性**（声明并响应式管理组件间依赖的能力）。我们把经典效应与共效应概念提升为运行时机制：形式化**可逆效应**——每个上下文变换携带运行时追踪的逆，追踪与恢复都保持组合性；形式化**响应式共效应**——上下文的每次变化按组件声明的规范通知组件为激活、停用或中性。两者统一进单一上下文类型，构成一个编程范式。再把机制组合为组件概念，给出动态组合演算，其元理论把时空可组合性从单个组件推广到整个交错组件系统。最后在**Cordis**——一个提供效应追踪与共效应解析核心库、声明式组件加载器（配置调和 + 热模块替换）的元框架——中实现这些思想，并以 **Koishi** 为案例研究。

关键词：动态组合；可逆效应；响应式共效应；插件系统；智能体脚手架

译注：本文为社区翻译版（非官方授权译本），数学符号保留原文 *Unicode* 形式；定理/引理/证明编号沿用原文；图 1、图 2 请参阅原论文 *PDF*（第 29、34 页）。原文：*A Programming Paradigm for Spatiotemporal Composability*, Yifan Shi (北京大学)、Wei Zhang (北京大学)、Tianyi Cui (DeepSeek-AI), 2026, 88 页, <https://github.com/cordiverse/paper>。

1 引言

组合——从简单部件装配出复杂系统——是软件工程的基本原则 [1]。传统上，组合是静态的：函数调用、模块导入和类继承在编译期解析，并在整个执行过程中保持不变。然而，现代软件日益需要 **** 动态组合 (Dynamic Composition) ****——组件在运行时被加载、卸载和重配置。插件架构 [2] 和自进化智能体脚手架 (Self-Evolving Agent Harnesses) 都要求系统能够在运行中安全地增删功能，但当前实践退而依赖粗粒度机制 [3]，只能通过重启来重配置，丢弃运行时状态。尽管动态组合的实际重要性不断增长，与静态组合已有的丰富形式化框架相比，其理论基础仍不成熟。

1.1 可组合性的两个维度

为了刻画动态组合的需求，我们在已被充分研究的组合代数性质之外，识别出两个正交维度：

- **时间可组合性 (Temporal Composability)** 对应时间维度：组件被移除时，它对其享环境所做的修改必须被完整且安全地逆转。这要求追踪组件执行的每一次资源分配、事件注册和状态变更，并保证移除时有序回收。
- **空间可组合性 (Spatial Composability)** 对应空间维度：组件必须能够以结构化、可验证的方式声明、发现并解析彼此之间的依赖。这要求管理依赖拓扑，并随依赖变化协调组件生命周期。

在静态场景下，时间可组合性退化为词法作用域（如 **RAII** [4]、括号模式 [5]），空间可组合性退化为模块导入解析 [6]。在动态场景下——组件在运行时到来和离开——两个维度都显著变难：时间可组合性必须处理作用域不受词法约束的长期有状态效应；空间可组合性必须处理在执行中出现、消失或改变身份的依赖。

1.2 动机示例

1.2.1 插件系统

插件系统是动态组合的典型实例。我们以 Visual Studio Code (VSCode) ——使用最广泛的可扩展 IDE 之一——为代表示例。

时间维度的局限。VSCode 在一个称为扩展宿主 (Extension Host) 的共享进程中运行所有扩展。虽然扩展可以动态安装，但该宿主不提供在运行时卸载单个扩展代码的机制。一旦扩展的 **activate** 函数执行完毕，停用或卸载它就要求重启整个宿主，影响所有已加载的扩展。纯声明式扩展（如主题、键位绑定、代码片段）不携带代码，可以自由移除；然而在安装量 **Top 100** 的扩展中，87 个包含可执行代码（数据取自 2026 年 6 月 9 日 VSCode Marketplace），卸载时必须重启。虽然 VSCode 提供了 **deactivate** 钩子，但它只是宿主进程终止时的优雅关闭回调，并不能实现热移除。此外，该钩子把效应的销毁与效应的创建（在 **activate** 中）分离，违反了关注点的局部性，使完整清理难以验证。

空间维度的局限。VSCode 确实提供了 `extensionDependencies` 用于声明扩展间的依赖, 但使用率极低: 安装量 Top 100 的扩展中, 只有 7 个对非内置扩展声明了 `extensionDependencies`。这种稀缺反映了扩展 API 的形态: 它暴露固定的、表层级的扩展点 (如命令、视图、语言特性), 扩展通过这些点向宿主贡献能力, 而非彼此依赖, 因此扩展间依赖很少出现。此外, VSCode 的扩展间交互机制没有结构契约: 它通过 `vscode.extensions.getExtension(...).exports` 向其他扩展暴露功能, 但返回值默认无类型 (`any`), 依赖方无法依赖受检查的接口。简言之, VSCode 引导扩展走向一组固定的宿主扩展点, 没有提供安全、结构化的方式让它们互相依赖。

这两个局限并非 VSCode 独有, 它们在插件系统中普遍复现 [2, 7], 只是程度不同。

1.2.2 自进化智能体脚手架

现代 AI 智能体依赖运行时智能体脚手架 [8-10]。这类系统可能组合多样化的工具套件 [11] 与执行环境、治理权限与沙箱、维护会话状态与持久化、提供上下文管理与记忆系统 [12]、编排子智能体与多智能体工作流 [13], 并向用户和自动化系统暴露接口。未来的脚手架可能在持续服务请求的同时, 生成并部署对自身组件的修改。模型合成的可复用工具是组件级自我修改的一个更窄的前身 [14]。每一次这样的修改本身就是一次动态组合。

由于这些修改持续发生且几乎没有人工监督, 动态可组合性变得不可或缺。没有时间可组合性, 每次自我修改都强制一次完全重启, 丢弃所有进程内累积状态; 在这种频率下, 累积不可用时间相当可观, 进行中的任务反复被打断; 更糟的是, 一次有缺陷的自我修改可能使恢复所需的进程本身失效。没有空间可组合性, 每个模块必须自行探测并适应其依赖模块的出现、消失或身份变化, 而只能依赖临时手段; 更糟的是, 朴素的代码替换策略可能静默地破坏依赖方, 或引入仅在重载时才暴露的循环依赖。

1.2.3 粗粒度的权宜之计

动态可组合性长期缺乏形式化关注的一个原因是: 操作系统和容器编排器已经提供了粗粒度的替代品。操作系统在进程粒度上提供时间可组合性; 容器编排器 [3] 在服务粒度上提供空间可组合性。实践中, 大多数软件通过退守这些粗粒度机制来容忍细粒度可组合性的缺失: 行为异常的模块靠重启进程处理, 服务依赖交由容器编排器管理。

然而这一权宜之计代价不菲。时间上, 每次重启丢弃所有进程内累积状态 (如缓存、连接、部分计算), 重建需要数秒到数分钟 [15]; 为在间歇期维持可用性, 需要冗余副本, 产生为弥补“无法恢复单个组件”而付出的资源开销。空间上, 容器级编排无法表达共享地址空间内组件之间的依赖, 并且对本来可以是本地函数调用的交互引入网络开销。两种机制都工作在进程与容器的边界, 而现代系统越来越多地在更细的粒度上组合。这种粒度错配要求一个与组件同粒度的组合抽象, 来管理效应与依赖。

1.3 贡献

动态可组合性的两个维度分别关涉: 计算如何**修改**环境, 以及计算如何**依赖**环境。这两个方向正是效应系统 [16, 17] 与共效应系统 [18, 19] 所形式化的内容: 效应提供推理环境修改的形式词汇, 共效应提供推理环境需求的形式词汇。然而, 现有形式化将推理局限于词法固定作用域上的编译期分析, 无法扩展到组件在运行时到来和离开的动态场景。通过把效应提升为可逆的运行时模型、把共效应提升为响应式的依赖解析机制, 我们获得动态可组合性的统一形式化基础——语言无关, 适用于任何需要动态组合的软件架构。本文做出如下贡献:

1. 形式化**可逆效应** (第 3.1 节): 每次上下文变换携带一个运行时追踪的显式逆, 追踪与恢复都保持组合性, 因此组件移除时上下文得以恢复。这建立了局部时间可组合性。
2. 形式化**响应式共效应** (第 3.2 节): 组件将其所需的共效应声明为规范, 上下文的每次变化按该规范通知组件为激活、停用或中性。这建立了局部空间可组合性。
3. 将效应上下文与共效应上下文统一为**单一上下文类型** (第 3.3 节), 其中共效应上的观察等价作为效应提供了独立性, 构成时空可组合性的编程范式。
4. 给出**动态组合演算** (第 4 节): 将两种机制组合为组件概念, 并为其生命周期配备操作语义。其元理论将时空可组合性从单个组件推广到整个交错的组件系统。

5. 在 **Cordis** 中实现这些思想 (第 5 节): 一个时空可组合性的元框架, 提供实现形式模型的核心库 (效应追踪与共效应解析), 以及带配置调和与热模块替换的声明式组件加载器。

2 预备知识

本节简要概述效应系统与共效应系统——本文工作的两大理论支柱。我们假定读者熟悉基础类型论与范畴论; 本节的目标是固定记号, 并引入第 3 节将操作化为运行时机制的关键抽象。

2.1 效应

在简单类型 λ 演算 (STLC) [20, 21] 中, 定型判断 $\Gamma \vdash t : T$ 表示项 t 在上下文 Γ 下具有类型 T 。效应系统细化类型以描述计算可能产生的副作用, 产生如下形式的判断:

$$\Gamma \vdash t : T \text{ effect} \quad (1)$$

这里, 结果类型标注了效应代数中的一个元素, 描述计算可能产生的副作用, 从而支持对有状态计算的组合式推理。这一方法起源于 Lucassen 与 Gifford [22], 他们引入了一个区分类型、效应与区域的 **kinded** 类型系统, 以发现并程序中的调度约束。

单子效应。Moggi [16] 首次通过单子对计算效应做了范畴建模; Wadler [23] 在 Haskell 中普及了这一方法。范畴 $\mathcal{C}$ 上的单子 (T, η, μ) 将有效应计算封装为类型 $T(A)$ 的值, 其中 $\eta : A \rightarrow T(A)$ 提升纯值, $\mu : T(T(A)) \rightarrow T(A)$ 对嵌套计算排序。经典实例包括 **Maybe** 单子 (部分性)、**State** 单子 (可变状态) 和 **IO** 单子 (外部交互)。

代数效应。Plotkin 与 Power [17, 24] 证明代数操作决定单子, 建立了一个效应接口与实现解耦的框架。效应签名 Σ 声明一组操作 (如状态操作的 $\text{get} : () \rightarrow S$ 、 $\text{put} : S \rightarrow ()$); 程序自由地调用操作, 而不承诺特定的解释。Plotkin 与 Pretnar [25] 随后引入效应处理器, 通过提供延续语义来解释操作:

$$\text{handle } e \text{ with } \{\text{op}(v, \kappa) \mapsto \dots\} \quad (2)$$

处理器接收操作参数 v 与定界延续 κ , 可以调用它零次、一次或多次, 从而在统一框架内支持异常、协程与非确定性 [26]。Koka [27, 28]、Eff [29]、OCaml 5 [30] 等语言已以不同的设计取舍采纳了代数效应。

2.2 共效应

与效应对偶地, 共效应系统 [18, 31] 丰富的是上下文而非类型, 产生如下形式的判断:

$$\Gamma \text{ coeffect} \vdash t : T \quad (3)$$

这里, 上下文标注了共效应代数中的一个元素, 描述计算对其环境的需求——如要访问的资源、要持有的权限、要依赖的服务。如果说效应刻画程序对世界的影响, 那么共效应刻画世界对程序的约束。

余单子共效应。用余单子构造上下文相关计算的想法最早由 Uustalu 与 Vene [32] 提出, 他们提出对称 (半) 么半余单子作为 Moggi 单子效应框架的对偶, 刻画数据流与属性求值等概念。Petricek 等人 [18] 在此基础上提出共效应, 作为上下文依赖的统一静态分析。余单子 (D, ε, δ) 刻画上下文相关计算: $\varepsilon : D(A) \rightarrow A$ 从上下文中提取当前值, $\delta : D(A) \rightarrow D(D(A))$ 为嵌套访问复制上下文。**Environment** 余单子 $D(X) = E \times X$ 建模对固定环境 E 的依赖; **Stream** 余单子 $D(X) = \mathbb{N} \rightarrow X$ 建模对时序数据的依赖。

分级共效应。为了更细粒度的追踪, 分级共效应系统使用预序半环 $S = (S, \leq, +, \times, 0, 1)$ 作为共效应代数 [33], 这一规范后来由 Gaboardi 等人与分级效应统一 [19]。 S 的元素标注每个变量绑定以量化其使用: 0 表示未使用, 1 表示线性使用, n 表示有界使用, ∞ 表示无限制使用。半环运算按顺序 ($\times$) 与并行 ($+$) 组合共效应, 在统一代数框架 [37] 内实现精确的资源追踪、敏感性分析 [34] 与信息流控制 [35, 36]。

2.3 与动态可组合性的关系

效应系统与共效应系统沿两个互补方向组织关于计算的推理：效应描述计算如何修改其环境，共效应描述计算如何依赖其环境。这两个方向正对应第 1 节识别的动态可组合性两个维度：

- 时间可组合性要求组件对其享环境的修改在卸载时可逆。相关效应是有状态的效应——它们持久地变换环境；撤销这种变换要求它存在逆。
- 空间可组合性要求组件间依赖被声明并响应式管理。这种依赖正是共效应所刻画的内容，而管理它们就是逐一按环境提供的内容进行解析。

然而，经典效应与共效应系统是静态工具：效应在词法固定作用域内被追踪，由编译期处理器释放；共效应标注在执行前确定的上下文上被验证。相比之下，动态组合要求这些保证对运行时到来和离开的组件成立，面对持续演化的上下文。没有固定的词法作用域能界定部署后才加载的插件；没有编译期上下文能预知运行时配置涌现的依赖。

这促使视角的转变：与其用更多标注扩展静态类型系统，不如把效应与共效应的概念结构 **** 具体化 (Reify) ****，使运行时能直接操作它们，从而在动态场景下建立这些系统在静态场景下所提供的保证。

3 可逆效应与响应式共效应

本节将第 2 节引入的效应与共效应概念提升为运行时机制，构建动态组合理论。核心思想是：把承载效应和共效应的定型上下文转化为 **** 上下文类型 ****——即把上下文具体化 (Reify) 为一等实体的、可在运行时操作的类型。对于效应类型，我们将其建模为 **** 携带逆变换的上下文变换 ****，实现局部时间可组合性；对于共效应上下文，我们将其建模为 **** 携带依赖信息的类型 ****，实现局部空间可组合性。共效应上的观察等价随后为效应提供独立性。同时承载效应与共效应的统一上下文，其本身就构成一个编程范式。

3.1 可逆效应

时间可组合性是指在运行时加载和卸载组件，使得卸载时共享环境恢复到组合前的状态。这要求组件对环境所做的每一次修改既可追踪又可恢复。因此，我们把效应建模为类型 $\Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)$ 的函数：作用于当前上下文，它产生修改后的上下文以及一个显式逆。提供该逆使效应可被撤销，将其返回给运行时使效应可被追踪。我们称这种效应为 **** 可逆的 (Revertible) ****：通过在执行期间追踪和组合这些逆，完整的环境恢复成为一种结构性保证。

3.1.1 效应上下文

给定任意不纯函数 $f_{\text{impure}} : X \rightarrow Y$ ，我们将其转化为纯形式 $f : \Gamma \times X \rightarrow \Gamma \times Y$ ，其中 Γ 是上下文，所有可能的副作用都可以表示为 Γ 上的变换。对任意固定输入 $x : X$ ，诱导映射 $\gamma \mapsto \text{pr}_1(f(\gamma, x))$ 独立于返回值地刻画 f 的副作用。因此 Γ 上的效应生活在变换幺半群 $\Gamma \rightarrow \Gamma$ (以 $\circ$ 复合) 中，其中每个幺半群公理都有对效应性质的直接解读：

- **封闭性**：两个效应的顺序复合仍是效应；
- **结合律**：复合效应与括号方式无关；
- **单位元**： id_Γ (Γ 上的恒等函数) 是复合的单位元。

为了建模可撤销的效应，我们把每个变换 f 与撤销 f 的另一变换 g 配对，称 g 为 f 的左逆，全文简称为 **** 逆 ****。撤销是单侧的：对逆的要求是 $g \circ f$ ，而从不要求 $f \circ g$ 。变换对有自己的乘法：

定义 1 (扭复合). 定义上下文变换对的扭复合 (Twisted Composition) 为

$$(f_1, g_1) \circ (f_2, g_2) := (f_1 \circ f_2, g_2 \circ g_1) \quad (4)$$

与 $\circ$ 本身一样，左操作数在右操作数之后作用，逆则以相反顺序累积。它使 $(\Gamma \rightarrow \Gamma) \times (\Gamma \rightarrow \Gamma)$ 成为以 $(\text{id}_\Gamma, \text{id}_\Gamma)$ 为单位元的幺半群——变换幺半群与其反向幺半群的乘积，我们称之为 Γ 上的 **** 扭复合幺半群 $\mathfrak{T}_\Gamma$ ****。

为了在上下文内部追踪效应，引入以下定义：

定义 2 (效应上下文). 给定上下文 Γ , 其 **** 效应上下文 (Effect Context) **** 定义为:

$$\partial\Gamma := \Gamma \times (\Gamma \rightarrow \Gamma) \quad (5)$$

它可以理解为一对 (γ, φ) , 其中:

- $\gamma : \Gamma$ 是当前上下文状态;
- $\varphi : \Gamma \rightarrow \Gamma$ 是 **** 累积器 (Accumulator) ****——迄今已执行效应的逆的复合, 即把上下文恢复到初始状态的函数。

特别地, 初始效应上下文可表示为 $(\gamma_0, \text{id}_\Gamma)$ 。我们也写作 $\partial^2\Gamma = \partial\Gamma \times (\partial\Gamma \rightarrow \partial\Gamma)$, 塔式依此类推。

定义 3 (track 变换). 定义上下文函数对上的变换 track_Γ :

$$\text{track}_\Gamma : (\Gamma \rightarrow \Gamma) \times (\Gamma \rightarrow \Gamma) \rightarrow \partial\Gamma \rightarrow \partial\Gamma, \quad \text{track}_\Gamma = (f, g) \mapsto (\gamma, \varphi) \mapsto (f(\gamma), \varphi \circ g) \quad (6)$$

该变换把正向函数 f 与候选逆 g 转化为效应上下文 $\partial\Gamma$ 上的变换。将 $\text{track}_\Gamma(f, g)$ 应用于状态 (γ, φ) : 用 f 变换 γ , 并把逆 g 复合到 φ 上, 从而在上下文中追踪 f 的效应。

定理 1. 对每个 $(f, g) \in (\Gamma \rightarrow \Gamma) \times (\Gamma \rightarrow \Gamma)$ 有 $\text{pr}_1 \circ \text{track}_\Gamma(f, g) = f \circ \text{pr}_1$ (即下图交换)。

Proof. 对所有 $(\gamma, \varphi) \in \partial\Gamma$:

$$(\text{pr}_1 \circ \text{track}_\Gamma(f, g))(\gamma, \varphi) = \text{pr}_1(f(\gamma), \varphi \circ g) = f(\gamma) = (f \circ \text{pr}_1)(\gamma, \varphi)$$

□

定理 2. track_Γ 是从 $\mathfrak{T}_\Gamma$ 到 $\partial\Gamma \rightarrow \partial\Gamma$ 的么半群同态:

1. $\text{track}_\Gamma(\text{id}_\Gamma, \text{id}_\Gamma) = \text{id}_{\partial\Gamma}$;
2. 对所有 $(f_1, g_1), (f_2, g_2) \in \mathfrak{T}_\Gamma$, $\text{track}_\Gamma((f_1, g_1) \circ (f_2, g_2)) = \text{track}_\Gamma(f_1, g_1) \circ \text{track}_\Gamma(f_2, g_2)$ 。

Proof. (1) $\text{track}_\Gamma(\text{id}_\Gamma, \text{id}_\Gamma)(\gamma, \varphi) = (\gamma, \varphi \circ \text{id}_\Gamma) = (\gamma, \varphi)$, 单位元映为单位元。(2) 任取 $(\gamma, \varphi) \in \partial\Gamma$:

$$\begin{aligned} (\text{track}_\Gamma(f_1, g_1) \circ \text{track}_\Gamma(f_2, g_2))(\gamma, \varphi) &= \text{track}_\Gamma(f_1, g_1)(f_2(\gamma), \varphi \circ g_2) \\ &= (f_1(f_2(\gamma)), \varphi \circ g_2 \circ g_1) \\ &= \text{track}_\Gamma(f_1 \circ f_2, g_2 \circ g_1)(\gamma, \varphi) \end{aligned}$$

□

定义 4 (recover 变换). 定义 $\partial\Gamma$ 上的变换 recover_Γ :

$$\text{recover}_\Gamma : \partial\Gamma \rightarrow \partial\Gamma, \quad \text{recover}_\Gamma = (\gamma, \varphi) \mapsto (\varphi(\gamma), \text{id}_\Gamma) \quad (9)$$

该变换把恢复函数 φ 应用于当前状态 γ , 并把 φ 重置为恒等。在 $\partial\Gamma$ 上应用一系列效应 $\text{track}(f_1, g_1), \dots, \text{track}(f_n, g_n)$ 之后, **recover** 把上下文恢复到初始状态。

定理 3. 对每个 $(\gamma, \varphi) \in \partial\Gamma$ 和每个满足 $g(f(\gamma)) = \gamma$ 的对 (f, g) ,

$$\text{recover}_\Gamma(\text{track}_\Gamma(f, g)(\gamma, \varphi)) = \text{recover}_\Gamma(\gamma, \varphi) \quad (10)$$

Proof.

$$\text{recover}_\Gamma(\text{track}_\Gamma(f, g)(\gamma, \varphi)) = \text{recover}_\Gamma(f(\gamma), \varphi \circ g) = (\varphi(g(f(\gamma))), \text{id}_\Gamma) = (\varphi(\gamma), \text{id}_\Gamma) = \text{recover}_\Gamma(\gamma, \varphi)$$

□

设 $(f_1, g_1), \dots, (f_n, g_n)$ 从 (γ, φ) 起依次应用, 写 $\delta_0 = \gamma$ 与 $\delta_i = f_i(\delta_{i-1})$ 。由定理 5, 复合 $\text{track}_\Gamma(f_n, g_n) \circ \dots \circ \text{track}_\Gamma(f_1, g_1)$ 是扭复合 $(f_n \circ \dots \circ f_1, g_1 \circ \dots \circ g_n)$ 的 **track**, 且若每个 $g_i(\delta_i) = \delta_{i-1}$, 则 $(g_1 \circ \dots \circ g_n)(\delta_n) = \delta_0 = \gamma$ 。该对因此满足定理 7 在 γ 处的假设, 一次应用即得

$$\text{recover}_\Gamma((\text{track}_\Gamma(f_n, g_n) \circ \dots \circ \text{track}_\Gamma(f_1, g_1))(\gamma, \varphi)) = \text{recover}_\Gamma(\gamma, \varphi) \quad (11)$$

取 $(\gamma, \varphi) = (\gamma_0, \text{id}_\Gamma)$, 恢复把以这种方式到达的每个状态都带回 $(\gamma_0, \text{id}_\Gamma)$ 。满足 $g \circ f = \text{id}_\Gamma$ 的对在每个状态都满足假设。

恢复通过量 $\varphi(\gamma)$ 读取状态, 我们称 $\varphi(\gamma) = \gamma_0$ 为 $\partial\Gamma$ 中状态的 **** 健全不变量 (Soundness Invariant) ****。

3.1.2 可逆效应函数

前一节的 **track/recover** 模型把逆当作先验给定： $\text{track}_\Gamma(f, g)$ 在看到任何上下文状态之前就固定了 g ，因此一个 g 必须服务于效应被应用的每个状态。实践中，每个效应的逆并非先验可知：它必须由调用方在效应应用处提供。此外，**recover** 是“全有或全无”的：它不能选择性地撤销一个效应而保留其他效应。为同时解决这两个问题，我们在输入侧和输出侧都增强模型：

1. **输入侧**：不仅变换 Γ ，还一并返回逆函数，使逆在效应应用处被提供： $\Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)$ ，即 $\Gamma \rightarrow \partial\Gamma$ ；
2. **输出侧**：不仅变换 $\partial\Gamma$ ，还一并返回逆函数，使一个效应可以被撤销而其他效应保留： $\partial\Gamma \rightarrow \partial\Gamma \times (\partial\Gamma \rightarrow \partial\Gamma)$ ，即 $\partial\Gamma \rightarrow \partial^2\Gamma$ 。

定义 5 (效应函数). 定义效应函数 $\mathfrak{E}_\Gamma$ 与带见证的效应函数 $\mathfrak{E}_\Gamma^*$ ：

$$\mathfrak{E}_\Gamma := \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma), \quad \mathfrak{E}_\Gamma^* := (e : \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)) \times ((\gamma : \Gamma) \rightarrow ((\delta : \Gamma) \times (g : \Gamma \rightarrow \Gamma) \times ((\delta, g) = e(\gamma) \rightarrow g(\delta))) \quad (12)$$

其中 $e(\gamma)$ 产生一对 (δ, g) ：

- $\delta : \Gamma$ 是新上下文；
- $g : \Gamma \rightarrow \Gamma$ 是当前效应的逆函数。

$\mathfrak{E}_\Gamma^*$ 的元素按状态选择其逆，约束 $g(\delta) = \gamma$ 把该选择限定为在应用处撤销效应，其他位置对 g 不加约束。单一的满足 $g \circ f = \text{id}_\Gamma$ 的 g 同时满足每个状态处的约束，并通过 $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$ 诱导 $\mathfrak{E}_\Gamma^*$ 的元素——定理 11 将证明这是同态。

定义 6 (效应复合). 给定 $f, g \in \mathfrak{E}_\Gamma$ ，其 ****** 效应复合 ****** $f \diamond g$ 定义为：

$$f \diamond g = \gamma \mapsto \text{let } (\delta, s) = g(\gamma) \text{ in let } (\varepsilon, t) = f(\delta) \text{ in } (\varepsilon, s \circ t) \quad (13)$$

定理 4. 效应复合把 $\mathfrak{T}_\Gamma$ 的么半群结构搬运到 $\mathfrak{E}_\Gamma$ ：

1. $(\mathfrak{E}_\Gamma, \diamond)$ 是以 $\eta_\Gamma := \gamma \mapsto (\gamma, \text{id}_\Gamma)$ 为单位元的么半群；
2. 赋值 $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$ 是从 $\mathfrak{T}_\Gamma$ 到 $\mathfrak{E}_\Gamma$ 的么半群同态。

Proof. (1) 结合律与单位元律由 $\circ$ 的相应性质逐分量得到。(2) 写 $e_i = \gamma \mapsto (f_i(\gamma), g_i)$ ；则 $(e_1 \diamond e_2)(\gamma) = (f_1(f_2(\gamma)), g_2 \circ g_1)$ ，正是 $(f_1, g_1) \circ (f_2, g_2)$ 的像，且 $(\text{id}_\Gamma, \text{id}_\Gamma)$ 映到 η_Γ 。□

定理 5. 见证在效应复合下保持，且均匀逆在每个状态都有见证：

1. $\mathfrak{E}_\Gamma^*$ 是 $\mathfrak{E}_\Gamma$ 的子么半群；
2. 定理 10 的同态把每个满足 $g \circ f = \text{id}_\Gamma$ 的对送入 $\mathfrak{E}_\Gamma^*$ 。

Proof. (1) 单位元在 $\mathfrak{E}_\Gamma^*$ 中，因为 $\text{id}_\Gamma(\gamma) = \gamma$ 。封闭性：取 $f, g \in \mathfrak{E}_\Gamma^*$ 与任意 $\gamma \in \Gamma$ ，令 $(\delta, s) = g(\gamma)$ 、 $(\varepsilon, t) = f(\delta)$ ，故 $(f \diamond g)(\gamma) = (\varepsilon, s \circ t)$ 。由见证有 $s(\delta) = \gamma$ 、 $t(\varepsilon) = \delta$ ，故 $(s \circ t)(\varepsilon) = s(\delta) = \gamma$ 。(2) $g \circ f = \text{id}_\Gamma$ 给出每个 γ 处的 $g(f(\gamma)) = \gamma$ ，故这样的对的像在每个状态都有见证。□

正如 **track** 把 Γ 上的变换对提升到 $\partial\Gamma$ ，我们定义 **effect** 把 $\mathfrak{E}_\Gamma$ 提升到 $\mathfrak{E}_{\partial\Gamma}$ ：

定义 7 (effect 变换). 定义效应函数变换 effect_Γ ：

$$\text{effect}_\Gamma : \mathfrak{E}_\Gamma \rightarrow \partial\Gamma \rightarrow \partial^2\Gamma, \quad \text{effect}_\Gamma = e \mapsto (\gamma, \varphi) \mapsto \text{let } (\delta, g) = e(\gamma) \text{ in } ((\delta, \varphi \circ g), \text{track}_\Gamma(g, \text{pr}_1 \circ e)) \quad (14)$$

由于 $\text{effect}_\Gamma(e)$ 本身是 $\mathfrak{E}_{\partial\Gamma}$ ，其返回值就是按定义 8 升高一级解读的逆。该逆本身是交换效应两个方向所得之对的 **track**。普通追踪规则再次适用：撤销效应本身就是一种效应（用 g 变换状态），而撤销它的方法就是再次执行该效应——这正是 $\text{pr}_1 \circ e$ 所做的。因此该逆复合到它收到的累积器上，与 **track** 的规定完全一致。

定理 6. **effect** 保持 $\diamond$ 运算。即 $\forall f, g \in \mathfrak{E}_\Gamma$ ：

$$\text{effect}_\Gamma(f) \diamond \text{effect}_\Gamma(g) = \text{effect}_\Gamma(f \diamond g) \quad (15)$$

Proof. 取任意 $(\gamma, \varphi) \in \partial\Gamma$, 令 $(\delta, s) = g(\gamma)$ 、 $(\varepsilon, t) = f(\delta)$, 故 $(f \diamond g)(\gamma) = (\varepsilon, s \circ t)$ 且 $\mathbf{pr}_1 \circ (f \diamond g) = (\mathbf{pr}_1 \circ f) \circ (\mathbf{pr}_1 \circ g)$ 。则

$$\begin{aligned} (\mathbf{effect}_\Gamma(f) \diamond \mathbf{effect}_\Gamma(g))(\gamma, \varphi) &= ((\varepsilon, \varphi \circ s \circ t), \mathbf{track}_\Gamma(s, \mathbf{pr}_1 \circ g) \circ \mathbf{track}_\Gamma(t, \mathbf{pr}_1 \circ f)) \\ &= ((\varepsilon, \varphi \circ s \circ t), \mathbf{track}_\Gamma(s \circ t, (\mathbf{pr}_1 \circ f) \circ (\mathbf{pr}_1 \circ g))) \\ &= \mathbf{effect}_\Gamma(f \diamond g)(\gamma, \varphi) \end{aligned}$$

第一步在 (γ, φ) 与 $(\delta, \varphi \circ s)$ 处展开定义 12, 第二步用定理 5, 第三步收拢定义 12。 $\square$

定理 7. 设 $e \in \mathfrak{E}_\Gamma$, 写 $f := \mathbf{pr}_1 \circ e$, 并令 $e' := \mathbf{effect}_\Gamma(e)$, 正向映射 $f' := \mathbf{pr}_1 \circ e'$ 。则

1. $\mathbf{pr}_1 \circ f' = f \circ \mathbf{pr}_1$;
2. 对每个 $(\gamma, \varphi) \in \partial\Gamma$, 提升逆 $g' := \mathbf{pr}_2(e'(\gamma, \varphi))$ 与在那里的逆 $g := \mathbf{pr}_2(e(\gamma))$ 满足 $\mathbf{pr}_1 \circ g' = g \circ \mathbf{pr}_1$ 。

Proof. (1) 由定义 12, $f'(\gamma, \varphi) = (f(\gamma), \varphi \circ g)$, 其状态是 $f(\gamma) = (f \circ \mathbf{pr}_1)(\gamma, \varphi)$ 。(2) 即对 $g' = \mathbf{track}_\Gamma(g, f)$ 应用定理 4。 $\square$

定理 8. 设 $e \in \mathfrak{E}_\Gamma^*$, 写 $f := \mathbf{pr}_1 \circ e$ 。固定 $(\gamma, \varphi) \in \partial\Gamma$, 令 $(\delta, g) = e(\gamma)$, 并写 (Δ, g') 为 $\mathbf{effect}_\Gamma(e)$ 在 (γ, φ) 处的值。则

$$g'(\Delta) = (\gamma, \varphi \circ g \circ f) \quad (16)$$

状态被精确恢复。累积器也被恢复——等价于 $\mathbf{effect}_\Gamma(e) \in \mathfrak{E}_{\partial\Gamma}^*$ ——当且仅当 $g \circ f = \mathbf{id}_\Gamma$; 且在任何情况下 $(\varphi \circ g \circ f)(\gamma) = \varphi(\gamma)$, 故健全不变量得以保持。

Proof. 由定义 12, $\Delta = (\delta, \varphi \circ g)$ 且 $g' = \mathbf{track}_\Gamma(g, f)$, 故

$$g'(\Delta) = (g(\delta), \varphi \circ g \circ f) = (\gamma, \varphi \circ g \circ f)$$

其中用了 $g(\delta) = \gamma$ 。属于 $\mathfrak{E}_{\partial\Gamma}^*$ 要求它在每个输入处等于 (γ, φ) ; 取 $\varphi = \mathbf{id}_\Gamma$ 把累积器相等变为 $g \circ f = \mathbf{id}_\Gamma$, 而该条件反过来给出每个 φ 的累积器相等。最后 $(\varphi \circ g \circ f)(\gamma) = \varphi(g(\delta)) = \varphi(\gamma)$ 。 $\square$

因此, 仅当在 γ 处见证的逆在每个状态都撤销 f 时下方三角形才闭合, 所以 $\mathbf{effect}_\Gamma$ 并不把 $\mathfrak{E}_\Gamma^*$ 送入 $\mathfrak{E}_{\partial\Gamma}^*$ 。任何情况下都成立的是在 γ 处的一致: $\mathbf{recover}_\Gamma(g'(\Delta)) = \mathbf{recover}_\Gamma(\gamma, \varphi)$ ——这正是定理 7 对累积器所要求的全部, 所以撤销不会扰动恢复目标。

以与应用相反的顺序撤销效应无需额外条件, 因为每个逆都会遇到它自己的应用所产生的状态:

定理 9. 设 $e_1, \dots, e_n \in \mathfrak{E}_\Gamma^*$ 从 $(\gamma_0, \mathbf{id}_\Gamma)$ 起依次应用, 并以相反顺序撤销。则

1. 每次撤销都精确恢复其应用所针对的上下文状态;
2. 每个中间状态满足健全不变量。

Proof. 每一步要么是应用要么是撤销。应用把 (γ, φ) 带到 $(\delta, \varphi \circ g)$ 且 $g(\delta) = \gamma$, 故按定理 7 (其假设恰是 $\mathfrak{E}_\Gamma^*$ 的见证) 保持 $\varphi(\gamma)$ 。逆序撤销把每个逆交到它自己应用所产生的状态手上, 故由定理 15 该撤销精确恢复前一状态并同样保持 $\varphi(\gamma)$; 两个结论都不依赖逆所收到的累积器。 $\square$

3.1.3 效应的独立性

定理 16 覆盖在其自身应用产生的状态处撤销效应; 在其他任意状态处撤销则属于本小节。有两种情形需要后者: 其一, 在后续效应仍就位时运行逆——这正是从运行中的系统撤回一个组件的含义; 其二, 一个序列可能交错多个组件的效应, 各自持有自己的逆, 于是某个组件的逆被另一个组件的应用隔开。两种情形中, 逆遇到的都是被外来效应移动过的状态, 它是否仍撤销它本应撤销的内容是一个 ** 交换性 ** 问题: 必须交换的是——一个效应能执行的每个变换与另一个效应能执行的每个变换, 正向映射与产生的逆都包括在内。单一累积器解决不了任一情形: φ 是按其固定顺序一次运行全部逆的复合。

定义 8 (变换幺半群). 对效应函数 $e \in \mathfrak{E}_\Gamma$, 其变换幺半群 $\mathfrak{M}(e)$ 是由 e 的正向映射连同 e 产生的每个逆所生成的 $\Gamma \rightarrow \Gamma$ 的子幺半群, $\mathfrak{M}(e)$ 的生成元是那个生成集的元素:

$$\mathfrak{M}(e) := \langle \{\mathbf{pr}_1 \circ e\} \cup \{\mathbf{pr}_2(e(\gamma)) \mid \gamma \in \Gamma\} \rangle \quad (17)$$

由对 $(f, g) \in \mathfrak{T}_\Gamma$ 诱导的效应以 f 和 g 为其生成元——它在每个状态产生的逆都是 g 。

引理 10. 交换性由生成元决定，且 $\diamond$ 不扩大任何变换么半群：

1. 若 $\mathfrak{M}(e_1)$ 的每个生成元与 $\mathfrak{M}(e_2)$ 的每个生成元交换，则 $\mathfrak{M}(e_1)$ 的每个元素与 $\mathfrak{M}(e_2)$ 的每个元素交换；
2. $\mathfrak{M}(e_1 \diamond e_2) \subseteq \langle \mathfrak{M}(e_1) \cup \mathfrak{M}(e_2) \rangle$ 。

Proof. (1) 与 $\mathfrak{M}(e_2)$ 每个生成元交换的映射构成 $\Gamma \rightarrow \Gamma$ 的子么半群 (含 id_Γ ，且 $f \circ f'$ 在其中若 f, f' 在其中)。该子么半群按假设包含 $\mathfrak{M}(e_1)$ 的生成元，故包含 $\mathfrak{M}(e_1)$ 。固定 $f \in \mathfrak{M}(e_1)$ ，与 f 交换的映射同样构成包含 $\mathfrak{M}(e_2)$ 生成元的子么半群，故包含 $\mathfrak{M}(e_2)$ 。(2) 由定义 9， $e_1 \diamond e_2$ 的正向映射是 $(\text{pr}_1 \circ e_1) \circ (\text{pr}_1 \circ e_2)$ ，它在任一状态产生的逆是 $s \circ t$ (s 由 e_2 产生， t 由 e_1 产生)。故 $\mathfrak{M}(e_1 \diamond e_2)$ 的每个生成元都是两个生成元之集的复合。□

定义 9 (独立性). 效应函数 $e_1, e_2 \in \mathfrak{E}_\Gamma$ 是 **** 独立的 (Independent) ****，当

1. 一个效应的每个变换与另一个效应的每个变换交换： $\forall f \in \mathfrak{M}(e_1), g \in \mathfrak{M}(e_2). f \circ g = g \circ f$ ；
2. 任何一方的变换都不扰动另一方产生的逆： $\forall g \in \mathfrak{M}(e_2), \gamma \in \Gamma. \text{pr}_2(e_1(g(\gamma))) = \text{pr}_2(e_1(\gamma))$ ， e_1 与 e_2 交换亦然。

族 $(e_l)_{l \in L}$ 逐对独立，当每 $l \neq l'$ 的 e_l 与 $e_{l'}$ 独立。族可以重复同一效应函数，而要求一个效应与自身独立就是要求 $\mathfrak{M}(e)$ 交换。

在独立性下，逆可以在被后续效应移动过的状态处运行，且它在那里撤回的只是它自己的贡献：

定理 11. 设 $e_1, \dots, e_n \in \mathfrak{E}_\Gamma^*$ 逐对独立并从 γ_0 起依次应用。写 $f_i := \text{pr}_1 \circ e_i$ ，令 $\delta_i := f_i(\delta_{i-1})$ ($\delta_0 := \gamma_0$)，并令 $g_i := \text{pr}_2(e_i(\delta_{i-1}))$ 为 e_i 在其应用处产生的逆。固定 j ，写 $\delta'_i := (f_i \circ \dots \circ f_{j+1})(\delta_{j-1})$ 为省略 e_j 后序列的状态 (故 $\delta'_j = \delta_{j-1}$)。则对每个 $j \leq u \leq n$ ：

1. $\delta_u = f_j(\delta'_u)$ 且 $g_j(\delta_u) = \delta'_u$ ；
2. 每个 $i > j$ 的 e_i 在 δ'_{i-1} 处产生与在 δ_{i-1} 处相同的逆 g_i 。

Proof. (1) 第一式是对 u 的归纳。 $u = j$ 时即 $\delta_j = f_j(\delta_{j-1})$ (δ_j 的定义)。归纳步： $\delta_{u+1} = f_{u+1}(\delta_u) = f_{u+1}(f_j(\delta'_u)) = f_j(f_{u+1}(\delta'_u)) = f_j(\delta'_{u+1})$ ，中间等式是定义 19 条款 (1) 应用于 e_{u+1} 与 e_j ($u+1 > j$ ，二者是族中不同效应)。第二式：条款 (1) 把 g_j 携带穿过 e_j 之后应用的正向映射，剩下 e_j 的见证在它唯一成立的那个状态处使用： $g_j(\delta_u) = (g_j \circ f_u \circ \dots \circ f_{j+1})(\delta_j) = (f_u \circ \dots \circ f_{j+1})(g_j(f_j(\delta_{j-1}))) = \delta'_u$ ，最后的等式依赖于 $g_j(f_j(\delta_{j-1})) = \delta_{j-1}$ (定义 8 对 e_j 在 δ_{j-1} 处要求的见证)。(2) 由 (1)，状态 δ_{i-1} 是 $f_j(\delta'_{i-1})$ ，且 $f_j \in \mathfrak{M}(e_j)$ ，故定义 19 条款 (2) 对 e_i 与 e_j 给出 $\text{pr}_2(e_i(f_j(\delta'_{i-1}))) = \text{pr}_2(e_i(\delta'_{i-1}))$ 。□

条款 (1) 定位了逆所到达的状态：它正是同一序列在从未应用该效应的情况下会到达的状态，无论其后应用了什么效应。条款 (2) 定位了其他效应在那里持有的逆，两者合起来允许定理再次应用于更短的序列：

推论 12. 设 $e_1, \dots, e_n \in \mathfrak{E}_\Gamma^*$ 逐对独立并从 γ_0 起依次应用， $g_1, \dots, g_n$ 如上。在 δ_n 处按 $\{1, \dots, n\}$ 的任意排列顺序应用这 n 个逆，都到达 γ_0 。

Proof. 对 n 向下归纳。设排列以 j 开头。由定理 20(1)，在 δ_n 处应用 g_j 到达 δ'_n (省略 e_j 的序列所到达的状态)，由定理 20(2)，其余效应在那里产生的逆正是手头的 g_i 。该序列作为子族仍逐对独立，故归纳假设适用于它以及排列的其余部分；空序列到达 γ_0 。□

LIFO 顺序就是这样一个排列，定理 16 无需任何假设即可按该顺序撤销。独立性买到的是 **** 其他任何顺序 ****，以及随之而来的交错多个组件的序列——第 4.4.2 节将把它推广到整个系统的轨迹。

这些构造合起来构成了 **** 可逆效应 ****： $\mathfrak{E}_\Gamma^*$ 中的每个效应函数显式提供自己的逆，**effect** 在效应上下文 $\partial\Gamma$ 上追踪这些逆， $\diamond$ 运算在保持可逆性的同时复合它们。它们交付的是 **** 局部时间可组合性 ****——“局部”在于保证只针对单个组件自身的效应而言。我们以如下准则表述之：对组件应用的每个效应函数序列，累积器恢复到它开始的上下文 (定理 7)，而撤销该序列把每个逆交到它自己的应用所针对的状态手上 (定理 16)。加载组件就是应用这样一个序列并把其逆累积进 φ ；卸载组件就是应用 φ 。

该准则遗漏两件事，而它们都在多个组件登场时出现：不按累积器强加的顺序撤销，以及交错其他组件效应的序列。独立性交付了它们 (推论 21)，且它是作用于效应之上的条件而非构造的

性质——第 3.3.2 节给出满足它的规范，第 4.4.2 节从整个系统的轨迹读出保证。独立性不成立之处，顺序必须由别处承载：单个组件内由累积器承载（无论效应如何都按 LIFO 撤销，第 4.3.2 节），跨组件则由声明的共效应承载（把一个激活排在另一个之前，第 4.3.1 节）。

3.2 响应式共效应

空间可组合性是指组件能够声明彼此依赖，且系统能够在运行时解析、提供和撤回这些依赖。这要求每当共享上下文变化时重新评估依赖满足性：组件在其依赖可用时 **** 激活 ****，在依赖被撤回时 **** 停用 ****。因此，我们把组件的依赖建模为 **** 规范 (Specification) ****，并把上下文的每次变化按该规范分类为激活、停用或中性。按规范分类正是检测满足性变化的手段；响应分类正是驱动激活与停用的机制。我们称这种共效应为 **** 响应式的 (Reactive) ****：通过分类上下文变化并由此驱动激活与停用，正确的共效应排序成为一种结构性保证。

3.2.1 共效应上下文

传统控制反转 (IoC) 容器 [38] 通常把依赖建模为简单的键值映射。本小节把 IoC 形式化为与可逆效应协同的共效应上下文，为动态组合提供数学基础。

定义 10 (共效应上下文). 给定类型族 $\mathcal{V} : K \rightarrow \text{Type}$ ，共效应上下文定义为依赖偏函数类型：

$$\Sigma := (k : K) \rightarrow \mathcal{V}_k \quad (20)$$

其中 $\sigma : \Sigma$ 是有限偏函数，为每个 $k \in \text{dom}(\sigma) \subseteq K$ 指派一个 $\mathcal{V}_k$ 类型的值。记号：

- $\sigma(k)$ ：应用 ($k \in \text{dom}(\sigma)$ 时有定义)；
- $\sigma[k \mapsto v]$ ：在 k 处绑定 v 、其余与 σ 一致的表；
- $\sigma \setminus k$ ：限制 ($k \in \text{dom}(\sigma)$ 时有定义)；
- $k \in \text{dom}(\sigma)$ ：成员关系。

类型族 $\mathcal{V}$ 的使用确保每个依赖键 k 关联特定值类型 $\mathcal{V}_k$ ，为依赖访问提供静态类型安全。扩展与限制携带前提条件（由下述运算施加）：依赖不能提供两次（扩展要求 $k \notin \text{dom}(\sigma)$ ），缺失时不能撤回（限制要求 $k \in \text{dom}(\sigma)$ ）。前提被违反时以错误信号化，且不产生迁移。

定义 11 (get/set 运算). Σ 上的 get 与 set 运算定义为：

$$\begin{aligned} \text{get} : (k : K) \rightarrow \Sigma \rightarrow \mathcal{V}_k, \quad \text{get} = k \mapsto \sigma \mapsto \sigma(k) \\ \text{set} : (k : K) \times \mathcal{V}_k \rightarrow \Sigma \rightarrow \Sigma \times (\Sigma \rightarrow \Sigma), \quad \text{set} = (k, v) \mapsto \sigma \mapsto (\sigma[k \mapsto v], \lambda \sigma'. \sigma' \setminus k) \end{aligned} \quad (21)$$

其中 $\text{get}(k)$ 要求 $k \in \text{dom}(\sigma)$ 、 $\text{set}(k, v)$ 要求 $k \notin \text{dom}(\sigma)$ 作为前提。

值得注意的是， $\text{set}(k, v)$ 具有类型 $\mathfrak{C}_\Sigma^*$ ——正是共效应上下文上的效应函数。因此可以直接应用第 3.1 节的效应机制： effect_Σ 提供依赖注册的自动追踪与恢复。这正是响应式共效应与可逆效应之间的 **** 协同 ****：共效应运算就是效应，而效应是可逆的。

定义 12 (键上的共效应). 键 k 上的共效应是三元组 $(\mathcal{V}_k, \simeq_k, \mathcal{A}_k)$ ，其中 $\mathcal{V}_k$ 是定义 22 的值类型， $\simeq_k$ 是 $\mathcal{V}_k$ 上的等价关系（键 k 处的值按它比较，第 3.3.2 节）， $\mathcal{A}_k$ 是共效应运算集——绑定在 k 处的值提供给持有它的组件的运算。运算 $a \in \mathcal{A}_k$ 携带参数类型 X_a 与结果类型 B_a ，且只作用于值本身：

$$a : X_a \rightarrow \mathcal{V}_k \rightarrow \mathcal{V}_k \times (\mathcal{V}_k \rightarrow \mathcal{V}_k) \times B_a \quad (22)$$

前两个成分构成 $\mathcal{V}_k$ 上按定义 8 要求带见证的效应函数，第三个是结果。每个运算都被要求尊重 $\simeq_k$ 。运算通过其提升作用于共效应上下文：

$$a_\Sigma(x)(\sigma) := \mathbf{let} \ (v, g, b) = a(x)(\sigma(k)) \ \mathbf{in} \ (\sigma[k \mapsto v], \lambda \sigma'. \sigma'[k \mapsto g(\sigma'(k))], b) \quad (23)$$

在 $k \in \text{dom}(\sigma)$ 时有定义，其前两个成分是 Σ 上的效应函数。

3.2.2 规范与通知

访问不存在的依赖是运行时失败。组件应当只在它声明的所有依赖都存在时才激活，而不是乐观地访问并在缺失时失败。这提出两个问题：组件声明的依赖是否被联合满足，以及状态变化时系统应如何响应。共效应上下文 Σ 携带自然的观察结构使两者都可处理：对任意共效应规范 $d \subseteq K$ ，定义满足谓词：

$$\sigma \models d := \forall k \in d. k \in \text{dom}(\sigma) \quad (24)$$

该谓词可判定 ($\text{dom}(\sigma)$ 有限)。由于对 σ 的所有变更都经过效应函数 (其逆恢复之前的域)，满足性的变化在每个效应边界都可检测。这是响应性的代数基础：效应系统保证每次共效应变化都被观察到。

定义 13 (共效应规范). 共效应规范是：

$$\mathcal{D}_\Sigma := \text{Set}(K) \quad (25)$$

表示组件从环境声明的依赖集合。

定义 14 (通知分类). 给定共效应规范 $d \subseteq K$ 与状态 $\sigma, \sigma' \in \Sigma$ ，定义：

$$\text{notify}_d(\sigma, \sigma') := \begin{cases} \text{activating (激活)} & \sigma \not\models d \wedge \sigma' \models d \\ \text{deactivating (停用)} & \sigma \models d \wedge \sigma' \not\models d \\ \text{neutral (中性)} & \text{否则} \end{cases} \quad (26)$$

这良定义，因为 $\sigma \models d$ 可判定且所有状态迁移由效应函数中介。响应式不变量是：激活迁移触发组件效应的执行 (带完整效应追踪)，而停用迁移通过应用累积器触发恢复。这些迁移的精确操作语义依赖其与控制流的交互，将在第 4 节展开。

set 与 **notify** 合起来交付 **** 局部空间可组合性 ****——“局部”与前述同义，保证只针对单个组件自身的共效应。准则：组件只在满足其规范的状态激活，因此它从不读取缺失的绑定；上下文的每次变化都按该规范分类，因此满足性的丧失在发生处即被检测并驱动停用。两者都直接来自上述定义。

该准则覆盖共效应排序的一个方向而非另一个。若组件 A 提供键 k 且组件 B 声明 $k \in d_B$ ，则 B 只能在 A 激活并提供 k 之后激活，因为 $\sigma \models d_B$ 要求 $k \in \text{dom}(\sigma)$ 。反之不成立：卸载 A 从 $\text{dom}(\sigma)$ 移除 k 从而破坏 B 的满足性，但通知本身既不能在 B 自身拆卸所需的期间保持 k 可读，也不能把 A 的恢复推迟到 B 结束之后。把撤回排序在它引发的停用之后是对其他组件而非行动组件的条件，因此属于保证的全局形式——第 4.3.1 节提供所需的机制。

3.2.3 隔离与拦截

基础共效应上下文 Σ 建模扁平的依赖表。实践中，系统可能需要为不同组件把不同值绑定到同一逻辑依赖。本小节用两种机制扩展共效应上下文：**** 共效应隔离 **** (同一键在不同上下文中解析不同) 与 **** 共效应拦截 **** (依赖访问上的横切行为)。

定义 15 (实现方式). 上下文上的效应函数允许两种实现 (Realization)：

- **原地实现 (In-place)**：原地变更上下文并返回非平凡逆；后继与输入别名，恢复运行逆以撤销变更。
- **派生实现 (Derived)**：保持输入不变并返回从其派生的全新上下文，以恒等为逆；恢复即丢弃派生上下文。

定义 16 (带隔离的共效应上下文). 定义带隔离的共效应上下文：

$$\Sigma_{\text{iso}} := (K \multimap R) \times ((r : R) \multimap \mathcal{V}_r) \quad (27)$$

表示为一对 (ρ, σ) ：

- $\rho : K \multimap R$ 是 **** 隔离域表 ****，为每个被隔离的键指派域标识符；不在 $\text{dom}(\rho)$ 中的键解析到自己的域，故在那里写 $\rho(k) = k$ ($R \supseteq K$)；

- $\sigma : (r : R) \rightarrow \mathcal{V}_r$ 是依赖表——从域标识符到类型值的依赖偏函数。

两层映射结构把逻辑层与存储层解耦，使依赖访问具有上下文感知。访问键 k 时，系统先解析 $\rho(k)$ 得到域标识符 r ，再访问 $\sigma(r)$ 取实际值。

共效应隔离机制实质上实现了一个运行时特设多态系统：通过隔离域标识符，同一依赖键在不同上下文中可以解析到完全不同的值，且该多态可在运行时动态调整。与传统依赖注入相比，共效应隔离提供更细粒度的控制，可为特定组件定制隔离；**set** 仍是效应函数 ($\mathfrak{E}_{\Sigma_{\text{iso}}}^*$) 从而继承可逆性，而 **isolate** 无需如此——它派生上下文而非写入共享表。

定义 17 (带拦截的共效应上下文). 定义带拦截的共效应上下文与规范：

$$\Sigma_{\text{inter}} := ((k : K) \rightarrow \mathcal{M}_k) \times ((k : K) \rightarrow (\mathcal{M}_k \rightarrow \mathcal{V}_k)), \quad \mathfrak{D}_{\text{inter}} := (k : K) \rightarrow \mathcal{M}_k \quad (29)$$

上下文 Σ_{inter} 是一对 (ι, σ) ： ι 是安装在上下文自身的上下文携带元数据（默认空 ϵ_k ）； σ 把每个键 k 映射到从元数据 $\mathcal{M}_k$ 到值 $\mathcal{V}_k$ 的提供函数。规范 $d \in \mathfrak{D}_{\text{inter}}$ 携带组件声明的元数据，以 $\text{dom}(d)$ 为依赖集。每个键为其元数据配备幺半群 $(\mathcal{M}_k, \oplus_k, \epsilon_k)$ ：合并 $\oplus_k$ 结合，单位元 ϵ_k 为空元数据。

当声明规范 d 的组件访问键 k 时，系统求值 $\sigma(k)(d(k) \oplus_k \iota(k))$ ：组件声明的元数据与上下文携带的元数据 ι 合并，提供函数作用于结果。合并遵循每个键自己的语义（如标量字段覆盖、集合值字段并集），且 ****** 右偏 ******—— $\iota(k)$ 优先并可覆盖组件声明，让外围上下文无需修改组件即可约束其共效应的使用方式（如第 6.3 节）。

3.3 上下文范式

第 3.1 节与第 3.2 节各自作用在一个上下文上：前者作为效应的载体，后者作为共效应的载体——留待回答的是：同时承载两者的单一上下文是什么样。本节给出该统一的具体构造，从共效应装配出提供第 3.1.3 节所缺效应独立性的观察等价，并论证所得上下文类型自身构成一个编程范式。

3.3.1 统一上下文

定义 18 (上下文类型). 上下文类型 Γ_{∞} 定义为：

$$\Gamma_{\infty} := \mu\Gamma. \Gamma \times (\Gamma \rightarrow \Gamma) \times \Sigma \quad (31)$$

三个投影分别为：

- Γ ：当前上下文状态（递归）；
- $\Gamma \rightarrow \Gamma$ ：累积器，恢复本层的效应；
- Σ ：携带依赖信息的共效应上下文。

在此定义下，效应映射 $\mathfrak{E}_{\Gamma_{\infty}}$ 作用于自身，把 ∂ -塔统一为单一自相似类型。共效应上下文 Σ 被结构性地集成：依赖运算 (**set**, **get**) 作用于 Σ ，累积器追踪其逆转。由于 Σ 底层的类型族 $\mathcal{V}$ 不受约束，系统需要跨组件共享的任何状态都可以编码为具有适当值类型的依赖—— Σ 涵盖所有共享可变状态，而不仅是组件间依赖。组件与其环境之间的每次交互都经过这一单一实体。

层级组合. Γ_{∞} 的递归结构支持层级控制：父上下文聚合多个子层效应，形成树状控制结构，在保持模块性的同时实现统一的跨层管理。效应变换实现了字面的“插拔”隐喻：

- 加载组件 = 执行其效应（插入插头）；
- 卸载组件 = 恢复其效应（拔出插头，不影响其他运行中的组件）；
- 层级中不同层的组件独立可加载可卸载；父上下文聚合并管理其全部子级的效应，支持任意嵌套组合。

3.3.2 观察等价

第 3.1 节的恢复保证断言的是状态的相等（定理 7），这是一种理想化——物理状态无法按原样恢复。例如，**free** 把块还给分配器却无法恢复 **malloc** 之前的堆布局；生成式名字也不会被丢弃它的逆所恢复，因为下一次创建会抽出全新的名字 [39]。因此第 3 节的相等应 ****** 向上取模一个等价 $\simeq^{**}$ 来解读，而我们把 $\simeq$ 取为观察等价：当没有观察者能区分两个状态时，它们相关。

定义 19 (共效应上下文的等价). 两个共效应上下文相关, 当它们把相同键绑定到相关值; 上下文的两状态相关, 当其共效应投影相关:

$$\begin{aligned}\sigma \simeq \sigma' &:= \text{dom}(\sigma) = \text{dom}(\sigma') \wedge \forall k \in \text{dom}(\sigma). \sigma(k) \simeq_k \sigma'(k) \\ \gamma \simeq \gamma' &:= \sigma_\gamma \simeq \sigma_{\gamma'}\end{aligned}\quad (32)$$

其中 σ_γ 是 γ 的共效应投影 (定义 32)。

状态中没有键绑定的部分由此被遗忘, 而遗忘正是定理 7 能按 $\simeq$ 解读的原因: 上述例子的堆布局与生成式名字位于关系之外, 除非某个键绑定了它们。

定义 20 (不可区分性). 设 V 按定义 24 携带运算集 $\mathcal{A}$, 以 $\mathfrak{M}(a)$ 记每个参数 $x : X_a$ 上效应函数 $a(x)$ 的变换幺半群 (定义 17)。 $\mathcal{A}$ 上的一个测试 (Test) 是各幺半群 $\mathfrak{M}(a)$ ($a \in \mathcal{A}$) 生成元上的有限词, 每个字母作用于其前面字母留下的值; 其结果是沿途作为运算正向映射的字母产生的结果, 前提失败处未定义。值 $v, v' : V^{**}$ 不可区分 (Indistinguishable) **, 写作 $v \approx_{\mathcal{A}} v'$, 当 $\mathcal{A}$ 上的每个测试在两者处同时有定义或同时无定义, 且产生相同结果。

引理 13. 不可区分性是运算所尊重的最粗关系:

1. $\mathcal{A}$ 的每个运算都尊重 $\approx_{\mathcal{A}}$ (按定义 24 的意义);
2. 每个被 $\mathcal{A}$ 所有运算尊重的等价都包含于 $\approx_{\mathcal{A}}$ 。

因此每个可容许的 $\simeq_k$ 选择都包含于 $\approx_{\mathcal{A}_k}$, 而 $\approx_{\mathcal{A}_k}$ 本身可容许。

Proof. (1) 设 $v \approx_{\mathcal{A}} v'$ 且 $a \in \mathcal{A}$ 作用于某参数。在测试前加一个字母仍是测试, 故正向映射到达的值不可区分, 任一产生的逆从不可区分参数出发到达的值亦然; 单字母测试给出两处同定义或同无定义以及结果相等。(2) 设 R 是那样的等价且 $v R v'$ 。测试的每个字母是某运算的正向映射或产生的逆, 尊重性携带 R 穿过任一个, 保持到达的值相关且每个字母处结果相等。故每个测试在 v 与 v' 处一致。 $\square$

定义 21 (尊重等价). 映射 $f : \Gamma \rightarrow \Gamma$ 尊重 $\simeq$, 当

$$\forall \gamma, \gamma' \in \Gamma. \gamma \simeq \gamma' \Rightarrow f(\gamma) \simeq f(\gamma') \quad (33)$$

两个映射相关, 当它们在每个状态一致; $\partial\Gamma$ 中的两对相关, 当两个分量都相关:

$$f \simeq g := \forall \gamma \in \Gamma. f(\gamma) \simeq g(\gamma), \quad (\delta, g) \simeq (\delta', g') := \delta \simeq \delta' \wedge g \simeq g' \quad (34)$$

尊重 $\simeq$ 的映射即下降到 $\Gamma/\simeq$ 的映射; 被 $\simeq$ 关联的两个映射即在那里下降到同一映射的两个映射。效应函数两者都需要: 前者使其计算的状态在商上确定, 后者使其返回的逆在商上确定。

定义 22 (按 $\simeq$ 解读的定义 8). 按 $\simeq$ 解读定义 8: $e \in \mathfrak{E}_\Gamma$ 属于 $\mathfrak{E}_\Gamma^*$, 当 e 作为映射 $\Gamma \rightarrow \partial\Gamma$ 尊重 $\simeq$, 且写 $(\delta, g) = e(\gamma)$, 对每个 $\gamma \in \Gamma$:

1. $g(\delta) \simeq \gamma$;
2. g 尊重 $\simeq$ 。

取 $\simeq$ 为 Γ 上的相等即恢复定义 8。

引理 14. 按定义 37 解读 $\mathfrak{E}_\Gamma^*$ 时, 第 3.1 节断言的每个状态相等在把 $=$ 换成 $\simeq$ 后成立, 且从 $(\gamma_0, \text{id}_\Gamma)$ 可达的每个状态的累积器尊重 $\simeq$ 。

Proof. 累积器是逆的复合, 每个逆按定义 37(2) 尊重 $\simeq$, 而尊重 $\simeq$ 的映射的复合尊重 $\simeq$ (基例 id_Γ)。第 3.1 节的证明原样通过——尊重性正是携带关系穿过逆的东西: 从 $g_2(\delta_2) \simeq \delta_1$ 与 $g_1(\delta_1) \simeq \gamma$, 尊重性给出 $(g_1 \circ g_2)(\delta_2) \simeq \gamma$, 这是每个逆复合所走的一步; 定理 7 的健全不变量按该步读作 $\varphi(\gamma) \simeq \gamma_0$ 。 $\square$

定义 23 (运算的独立性). 运算 a 与 a' 独立, 当其提升在每对参数处作为效应函数独立 (定义 19), 且任何一方的变换都不扰动另一方产生的结果:

$$\forall x : X_a, g \in \mathfrak{M}(a'_\Sigma), \sigma \in \Sigma. \text{pr}_3(a_\Sigma(x)(g(\sigma))) = \text{pr}_3(a_\Sigma(x)(\sigma)) \quad (35)$$

a 与 a' 交换亦然。键 k 是 ** 交换的 (Commutative) **, 当 $\mathcal{A}_k$ 中任意两个运算独立 (运算也要与自身独立)。

定理 15. 不同键处的运算是独立的。

Proof. 设 a 在 $\mathcal{A}_k$ 中、 a' 在 $\mathcal{A}_{k'}$ 中且 $k \neq k'$ 。由定义 24, $\mathfrak{M}(a_\Sigma)$ 的每个生成元形如 $\sigma \mapsto \sigma[k \mapsto u(\sigma(k))]$ (u 是 $\mathcal{V}_k$ 上的映射)——要么是正向映射的提升, 要么是产生逆的提升; a' 在 k' 处同理。两个这样的映射交换 (各自只读写一个键且两键不同), 引理 18(1) 把交换性从生成元推广到两个么半群。第二条件: a_Σ 在 σ 处产生的逆与结果都由 $\sigma(k)$ 决定, 而 $\mathfrak{M}(a'_\Sigma)$ 的每个生成元保持 $\sigma(k)$ 不动。□

定义 24 (共效应中介的效应函数). 共效应中介的效应函数构成最小集 $\mathfrak{E}_\Sigma^A \subseteq \mathfrak{E}_\Sigma$: 包含单位元 η_Σ , 且对键 k 、运算 $a \in \mathcal{A}_k$ 、参数 $x: X_a$ 与成员族 $(e_b)_{b \in B}$, 下式再次为成员:

$$\sigma \mapsto \mathbf{let} (\delta, s, b) = a_\Sigma(x)(\sigma) \mathbf{in} \mathbf{let} (\varepsilon, t) = e_b(\delta) \mathbf{in} (\varepsilon, s \circ t) \quad (36)$$

每个阶段执行一个运算并按结果选择后续, 故参数可以依赖已获得的结果。

定理 16. 设 $e_1, e_2 \in \mathfrak{E}_\Sigma^A$, 且两者运算都出现的每个键都是交换的 (定义 39)。则 e_1 与 e_2 独立 (定义 19)。

Proof. 按定义 41 的构造归纳, $\mathfrak{M}(e_i)$ 位于 e_i 中出现的运算的生成元所生成的子么半群内。条款 (1): 由引理 18(1), 只需 e_1 中某运算的生成元与 e_2 中某运算的生成元交换。两运算位于不同键时是定理 40; 位于同一键时该键按假设交换。条款 (2): 取 $g \in \mathfrak{M}(e_2)$, 对 e_1 的构造归纳——单位元在每个状态产生 $\mathbf{id}_\Sigma$; 在阶段处, 运算的独立性逐生成元应用于 g , 在 $g(\sigma)$ 处再次产生 s 与 b , 故选择同一延续 e_b , 条款 (1) 把它运行的状态放在 $g(\delta)$, 归纳假设在那里再次产生 t 。因此该阶段在 $g(\sigma)$ 处产生 $s \circ t$ 。□

组件与其环境之间的每次交互都经过上下文, 且类型族 $\mathcal{V}$ 不受约束, 故系统可以把跨组件共享的每个位置绑定在自己的键上 (第 3.3.1 节)。组件的效应函数于是成为共效应中介效应函数沿共效应投影的提升, 独立性转移到该提升。第 3.1.3 节留下的假设由此得到满足, 随之而来的是整个组件系统的时间可组合性。

该分解划分的是计算中 **** 可交换的部分 **** 与 **** 顺序敏感的部分 ****。可交换部分由效应承载: 组件按任务所需顺序执行它们, 推论 21 按系统方便的顺序撤销它们, 组件之间互不约束。顺序敏感部分由共效应承载——运算不交换的键就是必须从效应外部强加顺序的键, 而强加顺序有两个位置: 组件内由累积器强加 (无论效应如何都按 LIFO 撤销, 定理 16); 跨组件由声明的共效应强加 (一方提供另一方声明, 提供先于声明的满足, 第 3.2.2 节)。可组合性由此在组件粒度而非单效应粒度上获得——这正是第 4 节工作的尺度。

定理有两个边界值得指明。把每个共享位置绑定在键上是范式的 **** 规范 **** 而非构造的性质, 故系统无法具体化为共效应的位置位于第 6.1 节的边界之外, 也随之位于定理之外。而键的交换性是键所发布接口的性质, 满足它是对提供键的组件的义务, 而非消费它的组件。

3.3.3 范式定位

编程范式在如何处理副作用上有根本分歧。两个既定的极点界定了光谱:

显式状态串线 (函数式)。为保持引用透明性, 纯函数式语言把副作用建模为状态上的显式变换。State 单子 $S \rightarrow (A, S)$ [23] 把环境穿过每个计算。该方法产生强组合保证: 效应在类型中可见且适合等式推理。但代价是显著的人机工效成本: 调用链上的每个函数都必须接收并返回状态参数, 即便只是原样传递。随着效应维度增多 (日志、配置、I/O), 单子堆叠或效应处理器样板激增。

隐式变更 (命令式/OOP)。主流命令式语言允许组件在调用点无显式声明地修改共享状态并访问依赖。效应侧的代表是 React 的 `useEffect` 钩子: 它在组件的内部 `fiber` 上注册持久副作用, 但效应目标与注册机制都不作为显式参数出现——识别依赖隐藏运行时状态中的调用顺序位置。共效应侧, Java 的服务定位器模式 (如 Spring 的 `ApplicationContext.getBean(...)`) 在运行时从进程级注册表检索依赖, 每个调用点都需要空值检查与类型转换; 依赖关系隐式散布在整个代码库。理解 `f()` 如何修改或依赖系统需要传递式阅读其实现; 重构变得脆弱——移动或删除一个调用可能静默破坏远处的常量。

上下文范式结合了函数式方法的可追踪性与命令式方法的工效。效应与共效应都经过显式上下文参数中介。每个运算因此可归因于调用它的具体上下文, 进而归因于该上下文所属的组件。

除结合两极之长外，上下文范式让开发者逐个处理每个效应与依赖，并自动将其组合为系统行为。可逆效应一侧：开发者提供每个原子运算的逆，任何复合的逆由复合得出（第 3.1 节），于是组件的拆卸从其加载 **** 推导 **** 而来，而非与之并列书写。响应式共效应一侧：组件只声明它需要的依赖，运行时自动解析并重接线（第 3.2 节），在提供者被添加、移除或替换时保持接线一致。两个方向上，本需依赖开发者纪律的正确性成为范式的结构性质。

4 动态组合演算

第 3 节只在局部形式上建立了空间与时间可组合性。把它们推广到整个系统，需要把系统分解为组件：每个组件配对一个共效应规范与一个带见证的效应函数，使与共享环境的每次交互都可归因于其中之一。以下各节为该分解给出操作语义，并在全局形式上建立空间与时间可组合性。第 4.1 节与第 4.2 节呈现能让生命周期获得规则的最小演算——把每次迁移视为原子、即时且无谬的；第 4.3 节逐一放弃这三个假设（迁移可能运行的每个方向各放弃一次原子性），承认运行时在迁移起点与终点之间插入的控制流形式，到达真实运行时实际实现的演算；第 4.4 节建立该演算的元理论：保持性、全局时间与空间可组合性、进展性与汇合性。

4.1 组件与纤维

本小节固定规则所作用的对象：**** 组件 (Component) ****；**** 纤维 (Fiber) ****——组件的一个实例化，携带自己的生命周期状态；以及 **** 注册表 (Registry) ****——持有状态所携带的纤维，共效应上下文从中读出。

定义 25 (组件). 携带效应与共效应（定义 32）的上下文 Γ 上的组件定义为：

$$\mathfrak{C}_\Gamma := \mathfrak{D}_\Gamma \times \mathfrak{P}_\Gamma \times \mathfrak{E}_\Gamma^* \quad (37)$$

表示三元组 (d, p, e) ：

- $d : \mathfrak{D}_\Gamma$ 是定义 25 的共效应规范，声明从环境所需的依赖；
- $p : \mathfrak{P}_\Gamma := \text{Set}(K)$ 是 **** 提供 (Provision) ****，声明组件可以提供的共效应键； p 之外的任何键都不是其效应函数所写的；
- $e : \mathfrak{E}_\Gamma^*$ 是定义 8 的带见证效应函数，定义组件激活时贡献的效应连同撤回它们的逆。

两个声明是同一接口的两个方向： d 是组件从环境读取的， p 是组件写入环境的，而第 4.2 节不允许同一注册表的两个纤维提供相交。提供的不相交性正是本章与第 3.2.3 节分道扬镳之处：定义 28 的隔离允许一键经域表解析，使两个纤维可在不同域提供同一键。

定义 26 (纤维). 固定纤维名集合 $\mathfrak{N}$ 。实例化组件 $(d, p, e) \in \mathfrak{C}_\Gamma$ 的纤维是元组 $\langle d, p, e, \pi, \sigma, \tau, \theta \rangle$ ，其中：

- $\pi : \mathfrak{N} \cup \{\text{root}\}$ 是 **** 父指针 ****——本纤维实例化于其下的纤维，或根标记 root ；
- $\sigma : \Sigma$ 是纤维自己的共效应表（定义 22），激活前为空，运行中由其效应写入；
- $\tau : \{\perp, \top\}$ 是 **** 退休标记 ****——新纤维为 $\perp$ ，编排器退休该纤维后为 $\top$ ；
- $\theta : \Theta_\Gamma$ 是生命周期状态，在第 4.2 节的双状态模型中为 $\Theta_\Gamma := \text{Inactive} \mid \text{Active}(g, \omega)$ ，其中 $g : \Gamma \rightarrow \Gamma$ 是累积器， $\omega : d \rightarrow \mathfrak{N}$ 是 **** 承诺视图 (Committed View) ****。

承诺视图 ω 把纤维声明的每个键送到迁移提交时为它提供该键的纤维的名字。

定义 27 (注册表). 写 $\mathfrak{F}_\Gamma$ 为 Γ 上纤维的集合。状态 $\gamma \in \Gamma$ 携带注册表

$$F_\gamma : \mathfrak{N} \rightarrow \mathfrak{F}_\Gamma \quad (39)$$

一个有限偏函数，其父指针构成以 root 为根的树。记号： $\gamma(n)$ 即 $F_\gamma(n)$ ；状态明确时纤维字段以下标简写 $d_n, p_n, e_n, \pi_n, \sigma_n, \tau_n, \theta_n$ 。

纤维的名字赋予它跨自身变更而存续的身份：每条规则只重写一个纤维的生命周期状态而不动其他，因此规则必须指明是哪一个。

每个纤维拥有自己的表，意味着共效应上下文是 **** 派生 **** 而非存储的——它是激活纤维联合提供的内容：

$$\sigma_\gamma := \bigcup \{ \sigma_m \mid m \in \text{dom}(F_\gamma), \theta_m = \text{Active}(-, -) \} \quad (40)$$

并集良定义，因为纤维只写它声明的键 ($\text{dom}(\sigma_n) \subseteq p_n$) 且不同纤维的提供不相交，故每个 $k \in \text{dom}(\sigma_\gamma)$ 恰位于一个 Active 纤维的表中——该纤维的名字记作 $\text{provider}_k(\gamma)$ ，称为 k 的 **** 提供者 ****。

4.2 基础演算

定义 28 (目标视图). n 在 γ 处的目标视图把每个声明键映射到其提供者——是 $d_n \rightarrow \mathfrak{N}$ 的全映射；当 n 根本不应运行时为 $\perp$ ：

$$\text{target}_n(\gamma) := \begin{cases} \perp & \tau_n \vee \neg(\gamma \models d_n) \\ (k \in d_n) \mapsto \text{provider}_k(\gamma) & \text{否则} \end{cases} \quad (41)$$

状态 **** 静止 (Quiescent) ****，当每个纤维都到达其目标视图：

$$\text{quiet}(\gamma) := \forall n \in \text{dom}(F_\gamma). \begin{cases} \text{target}_n(\gamma) = \perp & \theta_n = \text{Inactive} \\ \text{target}_n(\gamma) = \omega_n & \theta_n = \text{Active}(-, \omega_n) \end{cases} \quad (42)$$

规则. 五条规则生成两类关系。编排规则（前缀 **O-**，写作 $\gamma \Rightarrow \delta$ ）是编排器可以执行的动作；其前提说明动作何时合法，而非何时发生。生命周期规则（前缀 **L-**，写作 $\gamma \longrightarrow \delta$ ）是系统在前提成立时无提示采取的一步。

$$\frac{n \notin \text{dom}(F_\gamma) \quad \pi \in \text{dom}(F_\gamma) \cup \{\text{root}\} \quad (d, p, e) \in \mathfrak{C}_\Gamma \quad \forall m \in \text{dom}(F_\gamma). p \cap p_m = \emptyset}{\gamma \Rightarrow \gamma[n \mapsto \langle d, p, e, \pi, \emptyset, \perp, \text{Inactive} \rangle]} \text{O-Insert}$$

$$\frac{n \in \text{dom}(F_\gamma)}{\gamma \Rightarrow \gamma[\tau_n \mapsto \top]} \text{O-Retire} \quad \frac{\tau_n = \top \quad \theta_n = \text{Inactive} \quad \forall m. \pi_m \neq n}{\gamma \Rightarrow \gamma \setminus n} \text{O-Remove}$$

插入与退休是仅有的外部输入：编排器请求纤维存在或停止存在，从不直接设置其生命周期状态。**O-Retire** 对纤维状态无条件，因为退休是请求，由生命周期规则执行。退休与移除分离同理：仍为 Active 的已退休纤维必须先停用，提前移除会丢弃累积器而泄漏。**O-Insert** 的最后一个前提正是单源纪律：编排器不得接纳声明同一键的第二个组件。

$$\frac{\theta_n = \text{Inactive} \quad \omega = \text{target}_n(\gamma) \neq \perp \quad e_n(\gamma) = (\delta, g)}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Active}(g, \omega)]} \text{L-Reload}$$

$$\frac{\theta_n = \text{Active}(g, \omega) \quad \text{target}_n(\gamma) \neq \omega \quad g(\gamma) = \delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Inactive}]} \text{L-Unload}$$

L-Reload 随逆一起安装承诺视图；**L-Unload** 应用逆并丢弃承诺视图。两者由同一比较驱动——这就是第 3.2 节的响应式纪律。

定义 29 (实例化原语). e_n 的一次应用（或第 4.3.2 节适用时的某次迭代）可以注册组件 $(d, p, e) \in \mathfrak{C}_\Gamma$ 。它取代状态映射而取该组件的 **O-Insert** ($\pi = n$)，并产生所注册纤维的 **O-Retire** 作为其逆。规则抽取名字（受 **O-Insert** 的新鲜性前提约束）并交给效应函数。

逆退休而非移除，原因在于逆必须在其被到达的任何地方都可应用。**O-Remove** 携带前提，由它构造的逆可能无法应用；**O-Retire** 的唯一前提是 $n \in \text{dom}(F_\gamma)$ 。

定义 30 (约束). 映射 $f : \Gamma \rightarrow \Gamma^{**}$ 约束于 (Confined to) $^{**}n$, 当对每个 $n \in \text{dom}(F_\gamma)$ 的 $\gamma \in \Gamma$, 写 $\delta = f(\gamma)$:

1. (写。) $\text{dom}(F_\delta) = \text{dom}(F_\gamma)$, 每个 $m \neq n$ 的 $\delta(m) = \gamma(m)$, 且 $\delta(n)$ 与 $\gamma(n)$ 只在 σ 上不同;
2. (读。) 在 σ_n 、每个 m 的 $\sigma_m|_{d_n}$ 以及无纤维表命名的状态部分上一致的两个状态, 被 f 携带到同样三点一致的状态。

条款 (2) 解释了组件何以能读取它声明的值: 它们位于其提供者的表中。它不能读的是 d_n 之外的表或任何控制字段——这阻止组件按未声明纤维的生命周期状态分支。

4.3 进行中的迁移

定义 31 (扩展生命周期状态). 本节的 Θ_Γ 替换为:

$$\Theta_\Gamma := \text{Inactive}(\zeta) \mid \text{Reloading}(i, g, \omega) \mid \text{Active}(g, \omega) \mid \text{Unloading}(g, \omega, \zeta) \quad (43)$$

其中 $i : \mathfrak{C}_\Gamma^{\text{iter}*}$ 是剩余效应迭代器 (定义 51), g 是迄今构建的累积器, ω 是承诺视图, $\zeta : \{\perp\} \cup \Xi$ 是结果 (Unloading 携带其停用所走向的结果, Inactive 携带其已到达的结果)。

图 2 画出这些状态形成的生命周期 (见原论文 PDF 第 34 页): $\text{Inactive} \rightarrow \text{Reloading} \rightarrow \text{Active} \rightarrow \text{Unloading} \rightarrow \text{Inactive}$, L-Divert/L-Raise 从 Reloading 直达 Unloading。

4.3.1 撤回

定义 32 (被依赖). 纤维 n 在 γ 处 ** 被依赖 (Relied upon) ** , 当某个其他已安装纤维把一个键解析给它:

$$\text{relied}_n(\gamma) := \exists m \in \text{dom}(F_\gamma), k \in d_m. m \neq n \wedge \text{installed}_m(\gamma) \wedge \omega_m(k) = n \quad (46)$$

$$\frac{\theta_n = \text{Active}(g, \omega) \quad \text{target}_n(\gamma) \neq \omega}{\gamma \longrightarrow \gamma[\theta_n \mapsto \text{Unloading}(g, \omega, \perp)]} \text{L-Leave} \quad \frac{\theta_n = \text{Unloading}(g, \omega, \zeta) \quad \neg \text{relied}_n(\gamma) \quad g(\gamma) = \delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Inactive}(\zeta)]} \text{L-Unload}$$

L-Leave 记录停用决定而不执行它——纤维停止提供其共效应, 同时保留自己的承诺视图与他人的不动。L-Unload 应用累积器、丢弃承诺视图, 并按携带的结果离开 Inactive。它是演算中唯一应用累积器的规则。

排序的两半由形式的不同部分承载: 可见性一半由承诺视图承载 (L-Unload 最后一步才丢弃它), 排序一半由前提 $\neg \text{relied}_n(\gamma)$ 承载——称之为 ** 守卫 (Guard) ** , 它把 k 的撤回推迟到每个解析到 n 的消费者都已离开。定理 63 建立两者。

守卫是按绑定而非按纤维施加的。这类守卫通常死锁; 阻止死锁的是 Unloading 加 σ_γ 只对 Active 纤维取并集: 一旦 L-Leave 标记了 n , 其表离开 σ_γ , 于是没有目标视图还能命名 n , 每个承诺给 n 的消费者自身都在退出途中。

4.3.2 迭代

定义 33 (效应迭代器). 效应迭代器 $\mathfrak{C}_\Gamma^{\text{iter}}$ 与带见证的 $\mathfrak{C}_\Gamma^{\text{iter}*}$ 定义为递归类型:

$$\mathfrak{C}_\Gamma^{\text{iter}} := \mu \mathcal{I}. \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathcal{I}) \quad (47)$$

$e(\gamma)$ 产生三元组 (δ, g, o) : δ 新上下文; g 当前效应的逆; o 指示延续——Nothing 终止, Just(i) 提供下一次迭代。

迭代器在每个连续迭代之间提供一个边界——在那里上下文是迄今迭代所成, 累积器只恢复这些。在此意义上效应迭代器是具体化的定界延续 (主流语言以 yield 暴露的结构 [43]), 模型直接映射到生成器。

以迭代器替换原子效应函数后, 基础 L-Reload 分裂为轨迹经过的开始状态:

$$\begin{array}{c}
\frac{\theta_n = \text{Inactive}(\perp) \quad \omega = \text{target}_n(\gamma) \neq \perp}{\gamma \longrightarrow \gamma[\theta_n \mapsto \text{Reloading}(e_n, \text{id}_\Gamma, \omega)]} \text{L-Begin} \\
\\
\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad \text{target}_n(\gamma) \neq \omega \quad (\delta, h) = (\gamma, \text{id}_\Gamma) \vee i(\gamma) = (\delta, h, -)}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Unloading}(g \circ h, \omega, \perp)]} \text{L-Divert} \\
\\
\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad \text{target}_n(\gamma) = \omega \quad i(\gamma) = (\delta, h, \text{Just}(i'))}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Reloading}(i', g \circ h, \omega)]} \text{L-Iter} \\
\\
\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad \text{target}_n(\gamma) = \omega \quad i(\gamma) = (\delta, h, \text{Nothing})}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Active}(g \circ h, \omega)]} \text{L-Finish}
\end{array}$$

每次迭代按定义 52 把新产生的逆复合到累积器上 ($g \circ h$)，故累积器按后进先出顺序应用逆。

4.3.3 异步

前面的各层允许环境在各次迭代之间移动，并假设每次迭代本身瞬时完成——其启动与落地是一步。我们抽象地建模非即时性：迭代产生 $\text{Future}(A)$ 类型的值， Future 是不透明类型构造子，其定义性质是提交与解析之间外部状态可以变化。

该层增加的是 **** 惯性 (Inertia) ****：一旦启动，迭代必然落地，落地不可拒绝。飞行中目标视图翻转时，只有让迭代落地的 **L-Divert** 分支可用——迭代落地，纤维随后停用。因此该层不增加规则与类型；在 Γ 的粒度上惯性是其全部内容，形式为对宿主可取 **L-Divert** 分支的限制。

4.3.4 失败

设 Ξ 为错误集，把效应迭代器细化为迭代可以就地抛出而非产生三元组：

$$\mathfrak{C}_\Gamma^{\text{fail}} := \mu\mathcal{I}. \Gamma \rightarrow \text{Either}(\Xi, \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathcal{I})) \quad (49)$$

见证只约束 **Right** 情形——抛出无可撤销。

$$\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad i(\gamma) = \text{Left}(\xi)}{\gamma \longrightarrow \gamma[\theta_n \mapsto \text{Unloading}(g, \omega, \xi)]} \text{L-Raise}$$

L-Raise 先恢复后记录：纤维携带错误作为结果路由进 **Unloading**，迄今构建的累积器在那里应用，纤维以 $\text{Inactive}(\xi)$ 到达——什么都没安装。**L-Begin** 以 $\text{Inactive}(\perp)$ 为前提，故生命周期不会从错误结果重入——这是结果的实质：扣留效应函数已证明在其运行状态上不可靠的纤维，而非对环境原样重试。失败记录在纤维上而非传播给父纤维，故迁移失败的组件让兄弟组件继续运行——这是插件宿主想要的行为。

4.4 元理论

本节从十条规则（三条编排规则 + **L-Begin/L-Iter/L-Finish** + **L-Divert/L-Raise** + **L-Leave/L-Unload**）读出两个可组合性维度的全局形式，并加上只有整个系统才能被追问的性质：它总是到达目标所要求的配置，且该配置与静态装配产生的相同。

定义 34 (步索引). 以 t 索引步， γ^t 是前 t 步到达的状态， $\text{step}^t := r(n)$ 是 t 步取的规则 r 与作用的名字 n 。 n 的一个 **** 片段 (Episode) **** 是 installed_n^t 成立的最大索引区间 $[b, u]$ 。

每条规则都分解为 $\gamma^{t+1} = \text{edit}^t(\Psi^t(\gamma^t))$ ——状态映射 Ψ^t （迭代的前向映射或 **L-Unload** 处的累积器或 id_Γ ）与编辑 edit^t （控制字段改写）。写 $\gamma \approx \delta$ 当两状态除控制字段外全部一致。

引理 17 (写分离). 读表 1 与定义 48：每步 t 与 γ^t 处存在的所有纤维 m, n ：

1. $\sigma_m^{t+1} \neq \sigma_m^t$ 仅在步 t 作用于 m 时，写入位于 Ψ^t 内；
2. ω_n 只在 **L-Begin** 处产生、只在 **L-Unload** 处消失—— ω_n^t 在 n 的片段上恒定；

3. $\Psi^t = g_n^t$ 只在 *L-Unload* 处, 没有其他步把 g_n 应用于状态;
4. 安装状态只在 *L-Begin* 进入、只在 *L-Unload* 离开;
5. π_n, d_n, p_n, e_n 随条目产生且不再被写, τ_n 单调、只在 $\top$ 处被 *O-Retire* 写。

引理 18 ($\simeq$ -不变性). 设 $\gamma \simeq \gamma'$ 。则第 4.3 节的规则在 γ 作用于 n 可应用, 当且仅当在 γ' 作用于 n 可应用, 且两次应用到达的状态再次 $\simeq$ 相关。

引理 19 (等变性). 设 $\chi: \mathfrak{N} \rightarrow \mathfrak{N}$ 为双射, $\chi \cdot \gamma$ 为携带注册表 $F_\gamma \circ \chi^{-1}$ 的状态。则 $\chi \cdot \gamma$ 是状态, 且 $\text{step}^t = r(n)$ 把 γ^t 带到 γ^{t+1} 当且仅当 $r(\chi(n))$ 把 $\chi \cdot \gamma^t$ 带到 $\chi \cdot \gamma^{t+1}$ 。

引理 20 (残留条目). 称 n 在 γ 处 **** 残留 (Vestigial) ****, 当 $\tau_n = \top$ 、 $\theta_n = \text{Inactive}(\perp)$ 、 $\sigma_n = \emptyset$ 且无 m 有 $\pi_m = n$; 残留条目满足 $\gamma \approx \gamma \setminus n$ 。残留纤维对任何作用于 $m \neq n$ 的规则的前提观察毫无贡献 (除 *O-Insert* 抽取该名字或声明其键两种被移除所放宽的前提)。

4.4.1 保持性

定义 35 (良构). 注册表 F_γ 良构, 当对所有 $m, n \in \text{dom}(F_\gamma)$ 与所有 $k \in K$:

1. $\pi_n \in \text{dom}(F_\gamma) \cup \{\text{root}\}$;
2. $m \neq n \Rightarrow p_m \cap p_n = \emptyset$;
3. $\text{installed}_n(\gamma) \Rightarrow \omega_n$ 在 d_n 上全且取值于 $\text{dom}(F_\gamma)$;
4. $\text{installed}_n(\gamma) \wedge k \in d_n \wedge \omega_n(k) = m \Rightarrow \text{installed}_m(\gamma)$ 。

定理 21 (保持性). 若 F^t 良构则 F^{t+1} 良构, 无论步 t 应用哪条规则。

Proof. (1) 只有 *O-Insert* 与 *O-Remove* 写 π 或 $\text{dom}(F_\gamma)$; 前者以 $\pi_n \in \text{dom}(F^t) \cup \{\text{root}\}$ 为前提, 后者以 $\forall m. \pi_m \neq n$ 为前提。(2) *O-Insert* 的末前提即其加入纤维的条款。(3) 唯一写 ω_n 的规则是 *L-Begin*, 其前提 $\omega = \text{target}_n^t \neq \perp$ 使它在 d_n 上全且取值于 $\text{dom}(F^t)$ 。(4) 条款只能在某已安装纤维倒下、某 ω 被写、或某 ω 命名的纤维离开 $\text{dom}(F_\gamma)$ 处失败; 三种情形分别对应 *L-Unload* (其守卫 $\neg \text{relied}_n^t$ 恰好阻止) 与 *O-Remove* (被移除者未安装故按条款 (4) 不被任何 ω 命名) 与 *L-Begin* (写入的是 *Active* 提供者的 target_n^t)。□

4.4.2 时间可组合性

定义 36 (迭代器的独立性). 对 $i \in \mathfrak{C}_\Gamma^{\text{iter}*}$, 设 $\text{reach}(i)$ 为含 i 且对延续封闭的最小迭代器集, 并按定义 17 以 $\text{reach}(i)$ 中每个迭代器的前向映射与产生的逆为生成元读变换幺半群:

$$\mathfrak{M}(i) := \langle \{\text{pr}_1 \circ i' \mid i' \in \text{reach}(i)\} \cup \{\text{pr}_2(i'(\gamma)) \mid i' \in \text{reach}(i), \gamma \in \Gamma\} \rangle \quad (54)$$

两个迭代器 i, j 独立, 当按这些变换幺半群读定义 19 成立:

$$\forall f \in \mathfrak{M}(i), g \in \mathfrak{M}(j). f \circ g \simeq g \circ f, \quad \forall i' \in \text{reach}(i), g \in \mathfrak{M}(j), \gamma \in \Gamma. \text{pr}_{2,3}(i'(g(\gamma))) \simeq \text{pr}_{2,3}(i'(\gamma)) \quad (55)$$

且对 j 对称。

定理 22 (恢复精确性). 设步序列逐对独立, n 的片段在 b 开启, $u \geq b$ 位于其中, $t_1 < \dots < t_l$ 是 $[b, u]$ 中作用纤维不是 n 的索引。则

$$g_n^u(\gamma^u) \approx (\Psi^{t_l} \circ \dots \circ \Psi^{t_1})(\gamma^b) \quad (56)$$

即在 γ^u 处应用 n 的累积器, 得到 (除控制字段外) 同样这些步从 γ^b 产生的状态——恰如 n 从未开始。

Proof. 对 u 归纳。步 u 作用于 n 时: *L-Iter/L-Finish/落地 L-Divert* 处 $\Psi^u = \text{pr}_1 \circ i_n^u$ 且 $g_n^{u+1} = g_n^u \circ h$, 见证条件给出 $h(\Psi^u(\gamma^u)) = \gamma^u$, 故 $g_n^{u+1}(\gamma^{u+1}) \approx (g_n^u \circ h)(\Psi^u(\gamma^u)) = g_n^u(\gamma^u)$; *L-Leave/L-Raise/中止 L-Divert* 处 $\Psi^u = \text{id}_\Gamma$, $h = \text{id}_\Gamma$ 同理。步 u 作用于 $m \neq n$ 时: 独立性给出 $g_n^u(\gamma^{u+1}) \approx g_n^u(\Psi^u(\gamma^u)) = \Psi^u(g_n^u(\gamma^u))$ 。□

推论 23 (终结恢复). 设片段 $[b, u]$ 关闭 (无论 n 到达什么结果), 则 $\gamma^{u+1} \approx (\Psi^{t_l} \circ \dots \circ \Psi^{t_1})(\gamma^b)$ 。

4.4.3 空间可组合性

定理 24 (排序). 纤维只在依赖被提供处开始迁移:

$$\text{step}^t = L\text{-Begin}(m) \Rightarrow \gamma^t \models d_m \quad (58)$$

设 $[b', u']$ 是 m 的片段且 $\omega_m^{b'}(k) = n$ ($m \neq n$, $k \in d_m$), $[b, u]$ 是含 b' 的 n 的片段, t 遍历 $[b', u']$. 则

1. $\omega_m^t(k) = n$;
2. $b < b'$, 且若 $[b, u]$ 关闭则 $u' < u$;
3. $k \in \text{dom}(\sigma_n^t)$ 且 $\sigma_n^t(k) = \sigma_n^{b'}(k)$.

Proof. 第一断言是 L-Begin 前提 $\text{target}_m^t \neq \perp$. (1) 是引理 54(2). (2): $b' - 1$ 处的 L-Begin 写 $\omega_m^{b'} = \text{target}_m^{b'-1}$, 其值是提供者, 故 $\theta_n^{b'} = \text{Active}(-, -)$, 而 $b - 1$ 处的 L-Begin 留下 $\theta_n^b = \text{Reloading}$, 故 $b < b'$; 若 $u \leq u'$ 则 $u \in [b', u']$, 于是 installed_m^u 且 $\omega_m^u(k) = n$, 即 relied_n^u ——与 u 处的 L-Unload 矛盾. (3): $[b', u']$ 内 n 只能被 L-Leave 作用 (其 $\Psi^t = \text{id}_\Gamma$), 故 σ_n 恒定. $\square$

定理 25 (解析一致性). 设 n 的片段 $[b, u]$ 以 $\omega_n^b = \omega$ 开启. 则 θ_n 在片段初始区间 $[b, r]$ 上是 Reloading, 且迁移的每次迭代都对着同一解析 ω 运行:

$$\forall t \in [b, r]. \text{step}^t \in \{L\text{-Iter}(n), L\text{-Finish}(n)\} \Rightarrow \text{target}_n^t = \omega \quad (59)$$

纤维离开该区间时 ($r < u$) 恰有一:

1. $\text{step}^r = L\text{-Finish}(n)$ 且 $\theta_n^{r+1} = \text{Active}(-, \omega)$;
2. $\text{step}^r \in \{L\text{-Divert}(n), L\text{-Raise}(n)\}$, 片段在 $u > r$ 关闭且 $\gamma^{u+1} \approx (\Psi^{t_l} \circ \dots \circ \Psi^{t_1})(\gamma^b)$ (如推论 62).

4.4.4 进展性

定义 37 (优先关系). 注册表名字上的优先关系是

$$n \prec m := p_n \cap d_m \neq \emptyset \quad (60)$$

即 n 可能提供 m 声明的键. 定理 66 与定理 73 在 $\prec$ 无环的假设上建立.

定理 26 (进展性). 假设 $\prec$ 无环、每个 n 有 $\text{len}(e_n) \leq K$ 、名字集 N 有限, 且每步应用生命周期规则. 写 $S(n)$ 为作用于 n 的步数、 $V(n) := |\{t : \text{target}_n^t \neq \text{target}_n^{t+1}\}|$ 为其目标视图翻转次数. 则

1. (无死锁.) $\neg \text{quiet}^t$ 蕴含某生命周期规则在 γ^t 可应用;
2. (终止.) $S(n) \leq (K + 4)(V(n) + 1)$, 且 $V(n)$ 与 $\sum_n S(n)$ 均有限.

Proof. 无死锁: 按 θ_n^t 的四种形态逐一给出可应用规则; 若全部排除, 则存在 Unloading 纤维 m_0 , 沿 $\prec$ 链构造 $m_0 \prec m_1 \prec \dots$ ——链上每一步要么守卫释放 (L-Unload 可用), 要么找到下一 Unloading 纤维; 无环性与 $\text{dom}(F^t)$ 有限使构造终止. 终止: 以两个断言界定 $S(n)$ (目标恒定的极大区间上至多 $K + 4$ 步; 每次翻转要么消耗严格 $\prec$ -下方纤维的一步要么消耗 τ_n 的唯一一次写入), 无环性给出良基递归 $B(n) := (K + 4)(2 + \sum_{m \prec n} B(m))$ 界定 $S(n)$. $\square$

4.4.5 汇合性

定义 38 (支撑). 纤维 n 在 γ 处 ** 被支撑 (Supported) **, 当它未退休、注册它的纤维被支撑、且它声明的每个键由被支撑纤维提供. 支撑关系是

$$m \sqsubset n := m \prec n \vee \pi_n = m \quad (62)$$

良基时 (引理 68), 支撑集 A 定义为:

$$n \in A := \neg \tau_n \wedge (\pi_n = \text{root} \vee \pi_n \in A) \wedge \forall k \in d_n. \exists m \in A. k \in p_m \quad (63)$$

定理 27 (汇合性). 设步序列到达静止的 γ^T 且无失败纤维, 步逐对独立、每个组件在其提供上全量 (定义 69), A 如定义 67. 则

1. (典范形式。) γ^T (除归约撤回名字的条目外) 可由如下序列从 γ^0 到达: 取同样的编排步于原始顺序, 按线性化 $\sqsubset$ 的枚举 $n_1, \dots, n_k$ 依次取每个 n_i 的一个片段;
2. (汇合。) 从 γ^0 取同样编排步的任意两条这样的序列, 在引理 56 的重命名后到达被 $\simeq$ 与 $\approx$ 关联的状态。

Proof. (1) 关闭片段先行 (按 $\sqsubset$ -极大性归纳, 引理 72 删除每个片段及其注册的名字而不动终点); 编排步前移 (引理 71(2)); 片段按 $\sqsubset$ 线性化排序并连块 (引理 71 的换位使终点不变)。(2) 两条归约都到达同一典范序列——同名携带同 d, p, π , 注册树由组件的迭代器函数决定, 名字差异由引理 56 的双射匹配。□

失败被排除在陈述之外, 因为它是真实的发散源: 一步是否抛出取决于它运行所对的状态, 一个调度可能失败某纤维而另一个完成它——两个静止状态于是在该纤维的生命周期状态上不同; 但由推论 62, 它们在别的任何方面都不同 (失败纤维对状态的贡献为零)。

该定理许可把 Cordis 应用当作静态装配来推理: 添加组件、移除它、替换提供者、再撤回替换的编排器, 保证到达与一开始写下最终组合相同的状态。它同时划定了保证的边界: 它谈论的是 **状态 **, 而非系统沿途产生的发射——这是第 6.1 节在 (边界内追踪的) 获取与 (跨越边界的) 发射之间画的界线。

5 实现与案例研究

本节呈现 Cordis——把第 3 节的形式模型实现为实用的编程抽象。Cordis 是时空可组合性的元框架: 不同于面向特定领域的应用框架 (Web 路由、ORM、UI 渲染), 它不预设任何具体场景; 其唯一职责是提供通用的动态组合语义。实现分三层: (1) 核心库 (第 5.1 节) 直接实现效应与共效应系统; (2) 组件加载器 (第 5.2 节) 在核心之上扩展配置调和与热模块替换; (3) Koishi 等应用框架 (第 5.3 节) 在前两层之上构建领域功能。

表 2 汇总了理论构造与其运行时对应 (此处从略, 见原论文 PDF 第 54-55 页): $\Gamma_\infty \leftrightarrow \text{ctx}$ (一等上下文); $\mathfrak{E}_r \leftrightarrow \text{ctx.effect}(\text{callback})$; $\Sigma, \Sigma_{\text{iso}}, \Sigma_{\text{inter}} \leftrightarrow \text{ctx}[\text{@@store}], \text{ctx}[\text{@@isolate}], \text{ctx}[\text{@@intercept}]$; 纤维 $\leftrightarrow \text{fiber}$ (含 $\text{fiber.uid}/\text{fiber.inject}/\text{fiber.apply}/\text{fiber.state}/\text{fiber}.\theta$ 的 LOADING 即 Reloading、FAILED 即 Inactive(ξ)。

5.1 核心库

5.1.1 效应追踪

Cordis 中的每次上下文变更都经过单一原语 ctx.effect : 共效应提供、组件实例化以及一切其他变更上下文的操作都归结为一次 ctx.effect 调用, 故通过上下文执行的任何操作都被自动追踪并在组件卸载时恢复。操作上, ctx.effect 是 $\text{effect}_r^{\text{iter}}$ (定义 52) 的实现: 接收 $\mathfrak{E}_r^{\text{iter}}$ 类型的回调并提升到 $\mathfrak{E}_{\partial r}^{\text{iter}}$, 产生一个 dispose 闭包——调用即恢复该效应。

算法 1 (效应追踪) 核心结构:

```

async function execute(callback, guard)
  iter ← callback(); inverse ← id
  while guard()
    (value, done) ← await iter.next()
    if value then inverse ← value ◦ inverse
    if done then break
  return inverse
function effect(ctx, callback)
  armed ← true
  task ← execute(callback, () ↦ armed)
  async function dispose()
    if not armed then return
    armed ← false

```

```

    recover ← await task
    recover()
    ctx.dispose ← dispose ◦ ctx.dispose
    return dispose

```

引擎 **execute** 把回调作为效应迭代器驱动，把每步产生的逆折叠为单一复合；每步之前咨询调用方提供的守卫，守卫一旦跳闸，迭代停止，只剩迄今累积的逆——这是第 4.3.2 节的步边界中断。**ctx.effect** 是薄包装，添加两点：**** 自处置 ****（守卫报告 **armed** 标记，**dispose** 翻转 **armed**，同时中止飞行中的迭代并保证恢复至多触发一次）；**** 父级组合 ****（**dispose** 前置到外围上下文的累积逆 **ctx.dispose** 上——子效应的逆本身是父级上的效应，即 $\partial^2\Gamma$ 的递归结构）。组件层（第 5.1.3 节）复用同一 **execute**，以测试 **fiber.target** 稳定性的守卫替代 **armed**。

5.1.2 共效应运算

所有共效应运算作用于每个上下文携带的三个符号键槽：

- **@@store**: 值存储 $\sigma: (r: R) \rightarrow \mathcal{V}_r$ （域符号到类型值）；
- **@@isolate**: 域表 $\rho: \text{Map}(K, R)$ （共效应键到域符号）；
- **@@intercept**: 拦截表 $\iota: (k: K) \rightarrow \mathcal{M}_k$ 。

前两者组成两层解析 $k \rightarrow \rho(k) \rightarrow \sigma(\rho(k))$ 。**ctx.set(key, value)** 是 **ctx.effect** 调用（**set** 的类型是 $\mathfrak{C}_\Sigma$ ），继承自动追踪与恢复；安装与移除都调用 **notify** 向依赖组件传播变更。**notify**（算法 3）对每个活纤维测试变更键是否出现在其 **fiber.inject** 中且解析到同一域；若是则调用 **refresh** 按新状态重评估该纤维——这是定义 26 的响应式分类。

一个绑定只有在安装它的纤维 **ACTIVE** 期间才对依赖者可用，故 **refresh** 按**激活的提供者**而非存储本身解析每个声明键——这是定义 46 的“由……提供”关系，它让撤回在发生前一步就对依赖者可见：进入 **UNLOADING** 的提供者已停止提供，其依赖者算出不满足的目标视图并开始自身拆卸，而绑定仍全部就位。

隔离与拦截做结构相同的事：各自派生一个子上下文，只调整继承表的一个键，父级不动——恢复是隐式的，丢弃子上下文即足够。**ctx.isolate(key, realm)** 覆盖域映射（实现定义 29 的 **isolate**）；**ctx.intercept(key, metadata)** 把元数据合并进拦截表（实现定义 31 的 **intercept**），新元数据与上下文已有内容合并并优先。

5.1.3 组件生命周期

组件由 **ctx.use** 实例化为纤维。两个字段驱动算法：**fiber.parent**（构成组件层级—— Γ_∞ 的递归结构）与 **fiber.inertia**（飞行中异步迁移的句柄）。

算法 4 (组件实例化): 组件配对共效应规范 **component.inject** 与效应函数 **component.apply**。回调函数是父纤维中追踪的效应：执行时以 **refresh(fiber)** 启动子级生命周期；恢复时把子的目标设为 $\perp$ 并触发 **unload**——这是定义 47 的注册原语，回调即其 **O-Insert**、回调返回的闭包即其 **O-Retire**：实例化是父级的一个普通追踪效应，故卸载父级会级联到子级。

算法 5 (组件生命周期) 实现第 4.3.3 节的惯性状态机：

```

function refresh(fiber)
  target ← target(y, n)
  if target = fiber.target then return
  fiber.target ← target
  if fiber.inertia then return
  if target ≠ ⊥ then
    fiber.state ← LOADING
    fiber.inertia ← create_task(reload(fiber))
  else
    fiber.state ← UNLOADING
    fiber.inertia ← create_task(unload(fiber))
async function reload(fiber)

```

▷ 任何逆被调度前先退出服务

```

target0 ← fiber.target
fiber.committed ← resolve(fiber.inject)           ▷ 提交视图
recover ← await execute(fiber.apply, () ↦ fiber.target = target0)
fiber.dispose ← recover ◦ fiber.dispose
if fiber.target = target0 then
  fiber.state ← ACTIVE
  notify(fiber.ctx, provided(fiber))
else
  fiber.state ← UNLOADING
  fiber.inertia ← create_task(unload(fiber))
async function unload(fiber)
  await all(notify(fiber.ctx, provided(fiber)).map(f ↦ f.await())) ▷ 排空依赖者
  await fiber.dispose()
  fiber.committed ← ⊥
  if fiber.target = ⊥ then
    fiber.state ← INACTIVE
  else
    fiber.state ← LOADING
    fiber.inertia ← create_task(reload(fiber))

```

`reload` 与 `unload` 都是惯性的：迁移一旦进入就运行到完成，系统才响应目标-状态变化；完成时互链（`reload` 发现目标已变则链入 `unload`，`unload` 发现目标非 $\perp$ 则链入 `reload`）——这是第 4.3.3 节的互递归。三条线承载定理 63 的共效应排序：`reload` 在第 14 行提交解析视图而 `unload` 在所有逆运行后才丢弃它（纤维加载期间——含自身拆卸——读到相同绑定）；`refresh` 第 10 行在迁移任务创建前标记 UNLOADING（L-Leave：纤维停止提供，依赖者先于其任何逆被调度而重算）；`unload` 第 25 行等待每个被通知的依赖者到达 INACTIVE（L-Unload 的守卫）。等待位于整个恢复之前而非某个被等待的逆之内。终止性由定理 66 得出：纤维只等待已不再可满足的依赖者，提供者图按需遍历而非预先分析。

5.1.4 上下文访问

在反射式 API（`ctx.set/ctx.get`）之上，Cordis 分层了第二种更原生的扩展方式：****** 属性访问 ******——组件以 `ctx[key]` 形式访问共效应，如同上下文的原生结构。TypeScript 中以 Proxy 实现，`get` 陷阱中介每次访问（算法 6）：从访问上下文沿纤维链向上走，在第一个承诺视图绑定 `key` 的纤维处授权并返回该绑定；途中若遇到声明 `key` 而未提交的纤维（未加载）则访问失败；到根仍无声明则拒绝为未声明访问。Proxy 与裸 `ctx.get` 的区别：后者对存储查表、返回绑定值或空、从不失败；前者对访问纤维自己的视图解析并在使用点强制执行共效应规范 d ——读视图而非存储也正是定理 63 所依赖的（它让依赖在其消失所触发的拆卸期间保持可读）。

5.2 组件加载器

核心库为组件开发者提供命令式原语（`ctx.effect`、`ctx.use`、`ctx.set`）。应用编排者面对另一关注点：把既有组件装配成运行系统并在其生命周期内调整组合。组件加载器引入声明式配置层：编排者把期望组合指定为持久数据结构，加载器把该规范的变更翻译为相应的命令式纤维操作。

5.2.1 声明式配置

条目 (Entry) 声明单个纤维，记录：`id`（稳定标识，调和键）、`url`（组件模块 URL）、`isolate`（隔离标注）、`intercept`（拦截标注）、`config`（绑定进组件形成其效应函数 `apply` 的配置）、`disabled`（行政停用标记）。条目可作为忠实规范，因为支撑纤维的恰是条目所记录的——定义 67 的支撑集只读 τ, π, d, p ，条目给出全部四个。

条目构成 **** 配置树 ****——系统加载内容的权威记录。`@cordisjs/group` 把子条目列表作为配置加载为子组，`@cordisjs/include` 加载外部配置文件 (YAML/JSON) 嫁接为嵌套子树；两者都是建立于定义 47 注册原语上的普通组件。

**** 调和 (Reconciliation) ****：条目记录变更时，加载器增量调和而非整体推倒重建——其健全性由元理论供给：定理 73 使静止状态只依赖最终配置；定理 66 证明系统确实会静止；推论 62 使离去纤维的状态贡献为零；定理 63 允许条目一起实例化而无须编排器安排加载顺序（声明键尚未被提供的纤维在 `L-Begin` 处等待，其提供者离去的纤维先于它停用）——故加载器可并发加载模块。逐字段分派最小扰动操作：`id/url` 变更重建条目；`isolate` 重指派域（算法 7）；`intercept` 原地更新（拦截元数据读时咨询，无需重载）；`config` 交给组件自行决定如何应用（通常与旧值 `diff`，仅在实质变更时重载）；`disabled` 置位时卸载纤维、清除时重载。

5.2.2 热模块替换

热模块替换 (HMR) 把可逆效应模式应用于模块层：源文件变更时（典型于开发期）系统原地替换受影响模块而不重启进程。因为纤维已界定其组件的全部效应与共效应，本身是组件的模块可以仅通过纤维操作替换：处置旧纤维恢复组件安装的一切，从重载模块实例化的新纤维重新安装——因此 HMR 不需要开发者标注的接受边界（不同于 Webpack [46]/Vite [47] HMR）。`@cordisjs/hmr` 组件三阶段工作：

1. **模块分类**（算法 8）：以 `stash` 集（自上次重载内容变更的文件）与 `externals` 集（不可热替换、触发整机重启的模块）为输入，对依赖子图求不动点——某模块的导入之一被接受则接受，导入全被拒绝则拒绝，悬而未决（导入环）默认拒绝；
2. **陈旧条目检测**（算法 9）：过滤出依赖树触及已接受模块的组件条目，该树随后并入 `accepted`；
3. **事务性重载**（算法 10）：失效 `accepted` 模块缓存、备份每个被移除模块以支持回滚，然后按 `url` 重新导入每个陈旧条目的组件模块并换入新纤维；任何模块导入失败（如语法错误）则恢复缓存、以备份重建所有陈旧条目并抛出——系统永不进入半重载状态。

5.3 案例研究：Koishi

Koishi 是构建于 Cordis 之上的开源聊天机器人应用框架。四年开发积累了 4000+ 社区贡献插件 (IM 适配器、数据库驱动、管理控制台、终端用户功能)。其规模与多样性使它成为 Cordis 动态可组合性的代表性生产验证。

元框架的表达力与通用性。Koishi 作为服务端 bot 运行，每个功能都是第 5.1 节上下文原语之上的插件——Koishi 自身只贡献聊天机器人领域词汇。同一模型出现在完全不同的运行时：Koishi 的 Web 控制台是第二个独立的 Cordis 应用，其插件组合的是浏览器与 UI 的原语而非服务端的。这确立模型的两个性质：(1) 表达力——原语足以承载完整生产系统，宿主框架只供领域词汇；(2) 通用性——它固定效应与共效应如何组合而把其含义留给每个应用，既不预设特定领域也不预设特定运行时。

无认知负担的时间可组合性。第 1.2.1 节调研的插件系统不重启扩展宿主就无法卸载单个扩展的效应。Koishi 例行执行此操作：编排者从控制台停用插件而其效应原地撤回；开发期 HMR 引擎保存时重新应用编辑过的插件，同时保持系统其他部分的缓存状态与活连接。Cordis 使移除不仅可能而且对插件作者毫不费力——经上下文执行的效应被追踪、其逆自动复合（第 3.1 节），连无经验的作者也能获得有序清理而无须书写卸载路径。

开放生态上的空间可组合性。与第 1.2.1 节（插件间依赖大多缺席）相反，Koishi 生态呈现真实的依赖拓扑：IM 适配器提供消息平台访问、数据库驱动提供持久存储、功能插件声明其为共效应并访问。运行时重配置提供者（切换存储后端、重连适配器）只重激活解析依赖实际变化的依赖者；依赖不可用的插件保持停用直到其出现，不出错。案例研究证实的是该组合跨独立作者代码成立——插件与其依赖通常由不同作者撰写，除连接它们的共效应外无任何协调。

效度威胁。证据来自单一宿主语言的单一生态，无法把范式优点与其 TypeScript 实现或 Koishi 特定领域分开；且是观察性而非对照比较。案例研究确立的是存在性-采纳性结果而非定量结果；对照基线测量抽象开销与对开发者生产力的影响留待未来工作。

6 讨论

前述形式模型与实现引入动态可组合性的编程范式。本节考察范式如何扩展到更广的工程关切，并讨论设计张力与开放问题。

6.1 系统边界

第 3.1 节的每个效应携带逆，逆的内容由系统边界裁定。边界把系统运行所对的环境分为两部分：(1) 位置在 **** 内 ****，当系统能排他地修改它并恢复修改前的状态——其上的操作在 Γ 中被追踪、可被恢复；(2) 位置在 **** 外 ****，当任一能力失效——其上的操作充当 id_Γ ，既不追踪也不恢复。

共效应移动边界：共效应通过具体化外部位置来移动边界——把对该位置的全部访问限制在它提供的一组运算内，每个运算它都能供逆，于是原充当 id_Γ 的操作进入 Γ 被追踪与恢复。边界因此按 **** 位置 **** 而非按介质划分。例：一块内存区域，仅系统写它时在内、其他进程也写时在外；一个文件，仅系统可达时（私有路径下的暂存文件）在内、其他程序读写的路径在外。

获取与发射：触达边界外的操作一般分两阶段。(1) **** 获取阶段 ****：获得访问并在边界内安装记录——`open` 安装 `close` 移除的描述符，`malloc` 预留 `free` 释放的块，`fork` 启动 `kill` 终止的子进程。该记录同时是数据可离开的通道。(2) **** 发射阶段 ****：把数据推过通道（`write` 交给文件的字节、`send` 放上线路的数据报），推送充当 id_Γ 。两阶段落在边界两侧：获取在内，发射跨外。

扣留与补偿：必须从发射恢复的系统有两个办法——扣留发射直到产生它的状态确定持久（回滚恢复的输出提交问题 [48]）；或补偿 [49]——按应用提供的比定义 33 的 $\simeq$ 更粗的等价恢复状态的动作（删除已创建的文件、退还已扣的款项）。补偿动作按与逆相同的 LIFO 顺序复合，但元理论不转移：定义 60 的交换性是对 $\simeq$ 证明的，需对更粗等价重新建立。

6.2 服务多路复用

OSGi [50] 等动态组件平台围绕服务组织组合：提供者在接口下发布、消费者绑定的功能单元。Cordis 共效应模型呼应此概念——服务对应键背后的接口。单一服务可由多提供者实现，两种形式：(1) **** 排他绑定 ****：多实现共享一接口但至多一个绑定，切换需卸载一个提供者并加载另一个，瞬时扰动每个消费者的依赖；(2) **** 服务代理 (Service Broker) ****：接口的中央入口服务被后备提供者与消费者共同注入，多提供者共存、代理分派请求。代理吸收扰动：更新后备提供者时代理原地不动，消费者看不到依赖变化、不触发重载。

服务代理支撑三种能力：**** 负载均衡 ****（按可配置策略分发：轮询/最少负载/延迟加权）；**** 滚动更新 ****（升级服务实现归结为受控提供者迁移——新提供者注册、ACTIVE 后流量渐移、旧提供者卸载；把传统基础设施级操作变成应用级组合模式）；**** 跨进程调用 ****（每个进程宿主自己的 Cordis 上下文与本地提供者，协调组件把它们连接为远程提供者，经保接口的 RPC 中介——代价是跨进程调用有延迟且可能中途失败，故跨进程暴露的接口必须设计为异步契约）。

6.3 访问控制与沙箱

由独立组件装配的应用需要两种互补机制：(1) 约束组件可访问的依赖；(2) 把不可信代码与宿主环境沙箱化。

基于能力的访问控制：第 5.1.4 节的依赖访问机制已构成代理中介属性上的一种访问控制——组件只能访问它声明的依赖，未声明访问即抛错。这与基于能力的安全 [54-56] 结构相似：权威由持有引用而非环境权威授予。`inject` 声明充当能力请求，上下文代理充当能力中介。请求是静态声明的，故组件所需全部代理中介能力在运行前已知，编排器可在加载时审查批准。该中介经拦截机制推广为细粒度策略：访问控制元数据可上下文携带或组件声明（定义 30），提供者在依赖被调用时咨询它以决定是否允许——如文件系统依赖携带“组件可读写哪些路径”的元数据。拦截位于上下文而非任一方代码，编排器可运行时调整而不触发重载、不扰动依赖图。

沙箱化不可信组件：组件代码不可信时语言级访问控制不够——恶意组件能直达底层对象。沙箱要求超出语言级手段的执行边界：软件故障隔离 [57]、独立语言运行时、沙箱进程或虚拟化容器 [58]。无论机制如何，不可信组件在自己的沙箱上下文中运行，经桥接触达宿主提供的依赖——推广第 6.2 节的跨进程调用：同样的透明性论证使桥接访问与本地注入无异；宿主侧桥接是普通纤维，其能力可由上述访问控制衰减。

6.4 语言无关性与选择

Cordis 以 TypeScript 实现，但上下文范式语言无关——时空可组合性只由其两个可组合性维度定义，任何满足沿两维最低要求的语言都可实现。**时间维**：最基本要求闭包（逆必须作为值捕获）与运行时引入/收回模块代码的能力——受管运行时靠可驱逐模块注册表（Node.js），原生代码靠显式动态链接（dlopen/dlclose、LoadLibrary/FreeLibrary），WebAssembly 视嵌入器取其一。**空间维**：归结为依赖注入问题，两个层面——类型层面（Haskell typeclass、Rust trait、TypeScript module augmentation 让提供者从自己模块扩展上下文类型）与运行时层面（JavaScript Proxy、Python 描述符协议等透明中介原语；缺省时靠反射，代价是类型安全与开发体验）。元编程设施可同时供给两层（注解/装饰器、Rust 过程宏、Scala 宏、Zig comptime）。

6.5 相互依赖与组件粒度

响应式共效应模型中，依赖环只让涉事组件永久停用： A 要 B 提供的键、 B 要 A 提供的键，则两者满足谓词永不成立。不同于并发系统的死锁（依赖调度、须发生时检测），此条件仅凭依赖声明即可预测，运行时可在加载时报告。

实践中大多数表面相互依赖可分解为更细粒度组件消除环。例：服务器（提供网络接口）与访问控制器（执行授权策略）双向交互——单块设计使彼此依赖。分解为四组件：server-core、access-control-core、request-mediation（依赖双 core 对入站请求应用访问控制）、policy-management（依赖双 core 经服务器暴露策略修改）——core 之间互不依赖，环消除。分解原则上是总能做的（每个双向交互都可因子化为独立单向绑定），但组件数增加： n 个相互交互的组件，集成组件数可随 n 二次增长。这不影响正确性或运行时性能（组件轻量），且更细粒度有收益（用户只加载所需集成绑定）；但影响开发体验。缓解粒度代价是工程问题：包捆绑、约定接线、脚手架工具。

6.6 依赖类型化与版本化

形式模型中依赖链纯粹由键身份建立：提供键 k 的组件满足任何声明 k 的组件。类型族 $\mathcal{V}_k$ 在单一编译单元内保证类型级一致，但组件独立开发构建时失效——产生两个问题。**接口漂移**：提供者在版本间修改 k 关联接口，按旧接口编译的消费者仍声明同键——共效应层满足（ $k \in \text{dom}(\sigma)$ ）而运行时值不再符合消费者预期。**键冲突**：两个独立开发的提供者用同名 k 表示完全无关接口——消费者无兼容性检查地接受另一方的值。两者指向同一缺口：共效应模型只提供名义链接（键名）而无版本化或结构化链接。三种修补：**键命名空间化**（键空间 $K \rightarrow K \times P$ ， P 标识接口定义包——最直接但最耦合）；**Peer 依赖**（宿主语言包管理器强制版本约束——Cordis 当前方案；局限是依赖语义化版本规范的自觉遵守，且包管理器通常把每个依赖解析到单一版本）；**结构兼容**（把成员检查替换为兼容谓词——结构子类型化 [73]；难题在于语言无关地定义该谓词：记录类型简单、行为契约复杂、含受限量化的参数多态不可判定）。设计统一依赖模型保持共效应模型动态组合保证，仍是开放问题。

6.7 与语言和操作系统的协同设计

与语言协同：围绕上下文范式设计的语言可在两方面超越库——给上下文的语义与给效应/共效应的原语。可让上下文 ****** 再次隐式 ****** 而保留第 3.3 节语义（库实现把上下文当普通参数传递，组件可能经闭包或全局变量误触他人上下文——隐式化同时收获工效与安全）。也可让编译器知晓效应与共效应：(1) 效应用语法，编译器为整个迭代发出单一状态机、在帧中持有逆，替代每步闭包分配；(2) 共效应规范进入类型系统——依赖环编译期报告、依赖按类型结构比较（行类型 [28]）。

与操作系统协同：第 1.2.3 节的粗粒度替代品是进程/服务粒度。与范式协同设计的 OS 可支持细粒度组合：把组件声明的共效应规范变成它可触达的全部，并以共效应形式提供自身资源。这样的 OS 能供给第 6.3 节外推给语言外机制的沙箱（像 WebAssembly 模块实例化时从嵌入器接收导入 [76] 一样，把组件界定到其声明依赖）；能以自身能力提供共效应隔离与拦截；能以共效应提供自身资源——内存与文件描述符是直接候选（内核接口处的恢复追踪已有先例 [77, 78]）；可把第 6.1 节只能扣留或补偿的部分操作变得可逆（事务性持久写入可回滚 [79]，写时复制/不可变存储移指针即达旧状态 [80, 81]）。

7 相关工作

动态可组合性与若干既有研究领域相交。我们综述最相关的线索，并把贡献与每一条区分开来。

7.1 效应与共效应系统

单子效应系统：一类库把效应编码进现有通用语言的类型系统，表示为运行时执行的单子值——Scala 的 ZIO [82] ($\text{ZIO}[R, E, A]$)、TypeScript 的 Effect-TS [83] ($\text{Effect}<A, E, R>$)、fp-ts [84] (基于 Reader 的单子变换子)。两个特性使它们与 Cordis 分开：其一，追踪以单子嵌入为代价——程序只有写在效应类型内才获得追踪，而 Cordis 把效应追踪作为普通宿主代码之上的覆盖层；其二，需求以解释（已安装服务供应其运算）释放，服务撤回时其运算所执行的仍在原地——Cordis 把每个效应与逆配对，并随提供者来去重新解析需求。

代数效应作为能力：与本文最接近的扩展是 Brachthäuser 等人的 Effekt 语言，把效应类型重释为能力 [85, 86]——效应类型表达计算对其上下文的要求而非它可能产生的副作用。Cordis 与 Effekt 两处不同：(1) 目的上，代数效应让效应可见以支持模块化解释（一个运算多种处理器语义），Cordis 让效应可见以支持追踪与撤销（每个上下文变换配对逆）；(2) 场景上，Effekt 在类型层静态规训效应（能力二阶化、局限词法作用域，经 boxing 恢复一阶使用），Cordis 在运行时规训效应（目标为组件移除时的完整资源恢复）。

可逆效应语义：并行的一条线给效应以可逆语义而非解释语义。Heunen 等人 [87] 在可逆场景中建模副作用，把 Hughes 的箭头适配为 dagger 箭头与逆箭头，刻画可逆运算（序列化、可变存储）。这是与本文可逆效应最接近的形式化：都把每个效应与撤销手段配对而非经处理器释放。差异在可逆性居于何处、要求多少：Heunen 等人在指称/范畴场景中工作，可逆性由构造保证的全局性质——每个计算可逆、逆双侧、从范畴结构恢复；Cordis 在运行时追踪逆且要求更少——不要求整个计算可逆，只要求每个原子效应承认单侧逆，由调用方在应用处提供而非推导，任何复合的逆由复合得出。

分级类型作为统一效应与共效应：Orchard 等人 [88] 提出分级模态类型作为同时涵盖效应推理（分级单子）与共效应推理（分级余单子）的伞概念，在 Granule 语言中实现；后续工作把共效应扩展到命令式 Java 类语言 [89, 90] 与按值调用按推 (CBPV) [91]。所有这些都工作在类型层：效应与共效应是编译期在词法固定作用域上检查的静态标注。本文贡献与此分析正交：把同一对概念提升为运行时机制，使 Cordis 处理动态组合——时间撤回与空间依赖随已加载组件集的演化而重新解析，而非对固定程序文本一次性裁定。

7.2 编程范式

面向上下文编程 (COP) [92, 93]：以层（部分方法与类定义）为语言装备，按执行上下文在运行时激活/停用，使行为适应而基代码不点名其上下文依赖。COP 与 Cordis 都视上下文为一等、运行时可变的实体并动态激活/停用行为，但相似是名义上的：COP 中“上下文”指环境执行情境（位置、用户、模式），激活改变动态作用域范围内的分派；层既不追踪其诱导的副作用也不撤销，激活不受依赖满足性支配。Cordis 中上下文是中介效应与共效应的 Γ_∞ 实体：激活运行组件的可逆效应并由响应式共效应满足性驱动，停用整体撤销。COP 变换运行什么行为；Cordis 组合并撤销组件安装的效应与依赖。

面向切面编程 (AOP) [95, 96]：把横切关注模块化为切面——量化基程序中连接点的切入点与在每个点织入的建议。Cordis 处理同样的原本会散布各组件的情境行为问题，但其切面类比是共效应：多组件声明依赖的共享中介点，横切行为可在那里重塑而无须编辑任何一方。两范式在两条轴上不同：(1) 声明 vs 无感——AOP 切入点无感且量化，匹配任意连接点（代码不知被建议）；Cordis 把横切限制在每个组件声明的共效应内，其触达恰是声明的表面。这产生确定性与可追踪性：编排者可在配置层审查与治理组件的横切而无须读其源码。(2) 生命周期集成——Cordis 中横切变更由组件效应携带，组件卸载时撤销并响应式传播给依赖者，是动态组合模型内的一步；动态 AOP [97, 98] 也能运行时织入/解织，但作为独立操作，既不绑定组件生命周期也不触发被建议代码间的重解析。

7.3 时间可组合性

有状态前向迁移：一族系统以跨版本携带状态前移实现无停机替换组件。都遵守同一时序纪律：组件只在到达安全、无交互点时方可交换。Kramer 与 Magee 把该准则确立为静止性 (Quiescence) [51]，Vandewoude 等人后来放宽为扰动更小的宁静性 (Tranquility) [52]；本文的滚动更新模式 (第 6.2 节) 以卸载提供者前排空在途请求执行之。动态软件更新 (DSU) 以手写变换函数前移状态：Hicks 等人的 C 通用 DSU [99]、Stoye 等人的类型安全更新点 [100]、Hayden 等人的 Kitsune [101]；同一纪律扩展到持久状态 [102]；Erlang/OTP [15] 在进程层持同样立场 (code_change/3 迁移状态、受监督进程重启恢复故障)；JavaScript 的 HMR (webpack [46]、Vite [47]) 在模块层做同样的事 (module.hot/import.meta.hot API 前移状态)。与 Cordis 的模块替换 (第 5.2 节) 相比，这些方法更优雅地迁移内存状态——Cordis 从干净板撤销旧组件的追踪效应并重应用新组件，组件自身内存状态在重载中不存活 (除非置于更长命的依赖中)；把 DSU 式前向迁移分层在可逆效应之上是未来工作。Cordis 的方法在两处更通用：不需要 DSU 与 HMR 类的手写迁移函数；支持完全卸载组件并恢复资源，而非仅原地更新。

开发者手写恢复：第二族靠开发者手写的清理或补偿逻辑恢复组件效应。插件生命周期约定 (OSGi [50]、Eclipse 扩展点、IntelliJ 与 VSCode) 把清理委托给开发者写的卸载回调；命令模式 [103] 把操作与撤销方法封装为撤销/重做栈；Saga 模型 [49] 把长事务结构为每步配补偿动作；代数效应处理器可附加拆卸时运行的终结器 [104]；事件溯源 [105] 以追加补偿事件撤回状态。所有这些中逆是未强制执行的义务、与操作解耦——遗忘的逆静默泄漏资源 (第 1.2.1 节有实证记载)。React 的 useEffect [106] 最接近结构性地配对效应与逆——返回运行时在每次重执行前与卸载时调用的清理。其短板是可组合性：钩子只能组件或另一钩子的顶层调用，不在条件、循环或嵌套函数内；效应体不接受异步函数或迭代器。效应因此无法从其他效应装配或与控制流交错，复合逆无从推导。Cordis 效应无此限制：普通操作自由组合、可异步运行，只对每个原子效应要求手写逆，任何复合的逆由组合推导——装配既有效应无需书写任何逆。每个效应与其逆的结构性配对使完整恢复成为系统不变量而非开发者纪律问题。

静态作用域撤销：第三族由构造自动撤销效应，但把撤销限制在预先固定的作用域。软件事务内存 [107, 108] 记录读写日志使一组内存操作要么提交要么中止；可逆计算 (Landauer、Bennett 的热力学分析 [110, 111] 到 Janus 等可逆语言 [112]) 更进一步使整个计算的每步全局可逆。可逆进程演算把回溯建入语义本身：RCCS [113] 随每个进程携带记忆，当一步所引向的过去因果等价时允许取回该步；Phillips 与 Ulidowski [114] 为 CCS、ACP、CSP 一致地推导可逆算子而保持其前向操作语义。其因果一致性准则是 Cordis 恢复所循顺序的并发对应物 (累积器 LIFO 应用组件自身逆、第 4.3.1 节守卫推迟提供者撤回直到消费者停用，定理 63)。但触达由语义固定——执行过的每个动作都保持可撤销；而 Cordis 组件为每个原子效应供逆，其累积器把上下文带回其组合开始之处。线性类型 [115]、RAII [4]、Rust 所有权系统 [61] 把资源释放绑定到词法区域——各自静态固定撤销的作用域与触达；Cordis 不预先固定任何作用域：在组件生命周期上撤销任意上下文操作，并视词法资源管理为互补 (适合单组件内的本地资源)。

中介式回收：第四族在运行时控制的接口处记录组件的获取，从而无须组件自供逆地回收其所得。Nooks [77] 包装跨越 Linux 内核与其可加载扩展之间边界的每个调用，扩展触碰的内核对象经过对象追踪器，其记录告诉恢复管理器扩展失败时释放什么；影子驱动 [78] 从另一侧截取同样的调用；Akeso [116] 以编译器插桩获得记录，把内核执行划分为可嵌套恢复域，把故障请求连同依赖它的每个域一起回滚。回收因此由运行时维护的记录而非开发者记得写的清理得出——这一族是可逆效应最接近的系统级先例。与 Cordis 的差异在词汇与触达：平台固定什么可记录 (每内核对象类型的释放代码、每驱动类的影子、每插桩分配器的逆)，故组件只能持有平台已知如何释放的资源；Cordis 组件引入自己的效应并为每个原子效应供逆。回收同样以提交请求或同扩展重启为界，而 Cordis 在组件整个生命期上撤销并向依赖者传播移除。

7.4 空间可组合性

初始化时依赖接线：两个既有机制在初始化时接线组件。依赖注入框架 [38] (Spring [117]、Guice、Angular、Inversify) 在初始化时注入依赖；UI 框架上下文 (Vue.js provide/inject、React Context API) 沿组件树传递。有些支持动态作用域 (Spring 的 prototype/request 作用域、Angular 的层级注入器)，但都不响应式重解析：提供者运行时被替换或移除时，既有依赖者既不停用也不重新初始化，且无任何一家提供本文组件状态机式的生命周期管理。Cordis 的响

应式共效应供给之：满足谓词每次变化都触发生命周期迁移。

可用性响应的组件模型：与我们最接近的先例响应服务可用性。OSGi 的 Declarative Services 与 iPOJO [118, 119] 让组件声明提供/所需服务，运行时随服务出现与消失自动激活停用；iPOJO 的 Gravity 项目明确以自主运行时适应为目标，其 provide/require 模型直接预示 Cordis 的 ctx.provide/ctx.get 模式。R-OSGi [53] 经 RPC 透明扩展同抽象到分布式场景。所有这些系统经停用回调恢复，两处局限：回调手写（资源安全靠开发者纪律）；回调同步（拆卸需要与离去依赖的异步交换时无协议等待）。Cordis 的响应式共效应封闭两个缺口：停用撤销依赖者累积的效应，惯性 Unloading 状态把异步拆卸运行到完成再处理进一步变更。

值级响应性：函数式响应式编程 (FRP) [120] 及其现代化身 (SolidJS 信号 [121, 122]、Vue 响应式系统、Angular Signals) 以值级粒度传播变更——信号变化时导出计算被同步或在调度器下重求值。Cordis 的响应式共效应作用于组件级粒度，添加值级传播不建模的异步生命周期语义。同样的粒度差异反向作用于一致性：以依赖图固定的顺序在轮次内传播，让 FRP 要求导出计算不读更新与陈旧输入的混合（无毛刺性 [124]）；而 Cordis 没有轮次对应物（编排动作逐个到达），只保证单次迁移不跨两种共效应解析（定理 64）。两者互补而非竞争：Cordis 共效应本身可携带响应式值，组件只在其实际消费的部分上更新——把组件级响应性细化为跨越两层级的更细粒度响应式共效应。

8 结论

本文通过把经典效应与共效应概念提升为运行时机制，呈现动态可组合性的形式基础。可逆效应处理局部时间可组合性：每个上下文变换携带运行时追踪的逆，追踪与恢复都保持组合，组件移除时上下文得以恢复。响应式共效应处理局部空间可组合性：上下文每次变化组件按共效应规范被通知，每次变化分类为激活、停用或中性，共效应隔离改变声明键的解析、共效应拦截改变绑定的使用方式。我们把效应上下文与共效应上下文统一为单一上下文类型，其中共效应上的观察等价于效应提供独立性，构成时空可组合性的编程范式。把这些机制组合进组件概念，得到动态组合演算，其元理论把时空可组合性从单个组件推广到整个交错组件系统。我们把该范式实现为 Cordis 元框架：核心库提供效应追踪与共效应解析，声明式组件加载器提供配置调和与热模块替换。Koishi 案例研究在拥有 4000+ 社区插件的生产系统中验证了 Cordis 的设计。

在人类策划的插件生态之外，未来验证的一个有说服力的方向是 **自进化智能体脚手架** (第 1.2.2 节)——AI 智能体在几乎无人监督下持续生成并替换自身脚手架组件。在此场景应用 Cordis 将验证快速组件替换下的完整恢复时间保证，以及频繁拓扑变化下的依赖协调空间保证。这样的验证将展示该范式作为智能体脚手架与其他自主系统可恢复、可协调、持续自进化的基础的适用性。

参考文献

参考文献保持英文原样，见原论文 PDF 第 80-88 页 ([1] Parnas 1972 至 [124] glitch freedom 共 124 条)。